

## Full Length Article

## Fast sparse supervised learning framework with BLInex loss function

Tiantian Jiang , Guolin Yu \*, Jun Ma

School of Mathematics and Information Sciences, North Minzu University, 750021, Yinchuan, Ningxia, China

## ARTICLE INFO

## Keywords:

Extreme learning machine

Blinex loss

Sparse

Adaptive moment estimation.

## ABSTRACT

In this paper, a novel learning model, namely the  $L_p$ -norm sparse Blinex Twin Extreme Learning Machine (PBLTELM), is proposed, which is efficient and accurate for relatively large-scale data classification. The proposed framework incorporates three key innovations: the robust Blinex loss function is integrated to enhance generalization,  $L_p$ -norm ( $0 < p < 1$ ) sparsity constraints are adopted to approximate  $L_0$ -norm solutions while ensuring computational tractability, and a dual-layer optimization strategy that combines iterative weight updates with the adaptive moment estimation (Adam) algorithm is developed to address the resulting non-convex and non-smooth problem. Theoretical analysis is conducted to verify convergence to local stationary points, which ensures both computational efficiency and model accuracy and properties particularly critical for relatively large-scale applications. Comprehensive empirical evaluations are performed across diverse benchmarks, including a two-dimensional artificial dataset, the CMU facial expression dataset, 12 UCI datasets, and 7 relatively large-scale libsvm datasets, to assess the performance of PBLTELM. The results indicate statistically significant improvements in classification accuracy and computational speed compared with state-of-the-art methods, confirming that PBLTELM serves as a scalable and competitive solution for relatively large-scale classification tasks.

## 1. Introduction

With the continuous development of society, artificial intelligence has been widely applied in fields such as economy, education, and healthcare. As one of the core tasks of machine learning, pattern classification plays a pivotal role in the field of artificial intelligence (Ahmed et al., 2020a,b; Alshemmary et al., 2024; Oleiwi et al., 2024). A classic binary classification model is support vector machine (SVM) (Vapnik, 1999; Wang & Shao, 2024), whose basic form is a maximum-margin linear classifier defined in the feature space and can be formally formulated as a convex quadratic programming problem. The CSKB algorithm (Tang et al., 2021) is a robust cost-sensitive kernel method based on the Blinex loss function, which is asymmetric and enables flexible control over the misclassification costs of minority and majority classes. To effectively improve the performance of classification tasks, a generalized feedforward neural network model based on a neural network structure, named extreme learning machine (ELM), was presented by Huang et al. (2004). A fundamental characteristic of this model is that the learning parameters of hidden nodes (including input weights and biases) are randomly assigned without the need for adjustment. Compared with other traditional single-hidden layer feedforward network (SLFN) learning algorithms (Ren & Yang, 2018), ELM exhibits faster learning speed and better generalization performance.

Although SVM and ELM have been extensively researched and applied in the field of robust classification and regression, they still suffer from issues such as time-consuming computation, high memory consumption, and difficulty in ensuring prediction and classification accuracy during data processing. To address these challenges, numerous new schemes have been proposed to accelerate the training process, among which generalized eigenvalue proximal support vector machine (GEPSVM) (Dufrenois & Noyer, 2015) and twin support vector machine (TSVM) (Ding et al., 2014, 2017; Yu et al., 2023) have garnered widespread attention. Two non-parallel hyperplanes are searched for by GEPSVM through solving two related generalized eigenvalue problems. A large quadratic programming problem is decomposed into two smaller ones via TSVM; based on the idea of GEPSVM, two non-parallel hyperplanes are searched for such that positive (negative) samples are as close as possible to the positive (negative) hyperplane and far from the other, leading to theoretically faster operation speed. Based on the concept, Pin-TSVM (Yan et al., 2019) integrated the noise-resistant characteristics and efficient framework of Pin-SVM, with its robustness enhanced by minimizing intra-class variance. The CPin-GTBSVM algorithm, proposed by Wang et al. (2024a), employs a truncated pinball loss function to improve robustness against feature noise and label noise. A wave loss function designed by Akhtar et al. (2024) possesses both noise robustness and critical smoothness, from which wave support vector machine

\* Corresponding author.

E-mail address: [yuguolin@nmu.edu.cn](mailto:yuguolin@nmu.edu.cn) (G. Yu).<https://doi.org/10.1016/j.neunet.2026.108843>

Received 6 August 2025; Received in revised form 9 March 2026; Accepted 10 March 2026

Available online 18 March 2026

0893-6080/© 2026 Elsevier Ltd. All rights reserved, including those for text and data mining, AI training, and similar technologies.

(Wave-SVM) and wave twin support vector machine (Wave-TSVM) are derived. The adaptive moment estimation (Adam) algorithm is adopted via these models, and classification speed is significantly improved by eliminating matrix inversion. Si et al. proposed a robust twin support vector machine SLTSVM based on the symmetric Linex loss function (Xing et al., 2026), which effectively improves the generalization ability and noise robustness of the model via regularization and efficient iterative optimization. Xing et al. presented a robust LSTSV (RLS-TWSVM) (Si et al., 2023), which adopts an adaptive pinball loss function, solves the model via the iteratively reweighted least squares method, and analyzes the convergence and complexity of the algorithm, thus significantly enhancing the noise resistance and generalization performance of the model.

Additionally, inspired by TSVM, a twin extreme learning machine (TELM) model was proposed by Wan et al. (2017) based on the ELM framework. Similar to TSVM, a large QPP is decomposed into two smaller subproblems for solution by TELM and its relevant variants, whose operating speed is significantly faster than that of ELM. A fractional loss function satisfying Bayesian rules was constructed via Yuan and Yang (2021), and its robustness was verified. Meanwhile, combined with this loss function, a new robust twin extreme learning machine (LCFTELM) was extended to mitigate the negative impacts of noise and outliers. The idea of projection twin support vector machine (PTSVM) was integrated into the basic framework of TELM by Yang et al. (2023), and a new binary classifier, projection twin extreme learning machine (PELM), was presented for the first time. Two projection directions are searched for by this model in the TELM feature space to achieve effective separation between the projected samples of one class and those of the other class. At the same time, the capped  $L_1$ -norm distance metric is introduced into PELM, and the non-convex optimization problem is transformed into a quadratic programming problem via reweighting technology, which enhances the robustness of the model against outliers. A weighted linear loss function is adopted via weighted linear twin extreme learning machine (WLTELM) (Anand et al., 2023), making it insensitive to outliers. The solution process of WLTELM is divided into two steps, which can be completed by solving a system of linear equations, endowing the model with short training time and excellent generalization ability.

Against the backdrop of large-scale data processing, the integration of sparse optimization and robust optimization has emerged as a key trend in classification framework research. Significant progress has been achieved by researchers across multiple fields through reducing the number of support vectors and optimizing model sparsity, as sparse optimization not only helps improve classification performance but also significantly reduces computational costs. Based on this, a sparse SVM model combining a two-parameter pinball loss function for the insensitive zone, named sparse PinSVM, was designed in Huang et al. (2013). An improved sparse non-parallel support vector machine (SNSVM) based on TSVM was derived in Tian et al. (2014), whose performance is comparable to that of standard SVM, but may degrade to that of traditional TSVM with appropriate parameter selection. Further, sparse discriminative twin support vector machine (SDTSVM) was designed in Zheng et al. (2022), which significantly enhances classification performance and training efficiency by optimizing the discriminability and sparsity of TSVM. Additionally, the sparse robust support vector machine (r-SVM) model was extended in Wang and Shao (2023a) and Wang and Shao (2023b) by constructing a smooth truncated absolute deviation (SCAD) loss function, further improving the performance of TSVM on large-scale datasets. These innovations have not only improved the computational efficiency of TSVM but also enhanced its robustness on complex datasets. A support vector machine method with two-parameter pinball loss was developed in Sharma et al. (2021), which effectively addresses noise issues and significantly reduces training time and memory requirements. Sparse pinball twin support vector machine (SP-TSVM) based on the insensitive zone pinball loss function was proposed in Tanveer et al. (2021), Tanveer et al. (2019a), Tanveer et al.

(2019b), and Tanveer et al. (2022), which overcomes the noise sensitivity of traditional TSVM, ensures model sparsity, and maintains the same computational speed as the original TSVM. The PLSTKSVC algorithm (Moosaei & Hladik, 2023) is a method based on least squares twin multi-class support vector machine; it achieves sparse solutions by introducing the  $L_p$ -norm ( $0 < p < 1$ ) to approximate the  $L_0$ -norm and generates ternary outputs under the 1-versus-1-versus-rest structure  $\{-1, 0, +1\}$ . Classification and feature selection can be performed simultaneously, but the classification speed is relatively slow. Shen and Ma (2019) further introduced the  $\epsilon$ -insensitive zone pinball loss function into ELM to achieve both robustness and sparsity. A sparse model named STSP-TSVM was presented in Wang et al. (2024b), which controls the number of support vectors through sparse constraints, thereby enabling fast classification computation on large-scale datasets. For Pin-GBTSVM (Quadir & Tanveer, 2024), granular balls rather than individual data points are used as the input, the core information of multiple internal data points is encapsulated by each granular ball via its center, only representative granular balls are retained for classifier construction, and the number of effective samples actually involved in computation is significantly reduced, thus achieving sparsity in terms of data input.

Thus, to achieve fast and accurate classification in large-scale data processing, this paper proposes a method based on the TELM framework. Via introducing the BLinex loss function, adopting the  $L_p$  norm ( $0 < p < 1$ ) to approximate the  $L_0$  norm for obtaining sparse solutions, and utilizing iterative optimization of weight vectors along with the Adam algorithm to solve non-convex and non-smooth optimization problems, high speed and sparsity in classifying large-scale datasets are realized. The specific contributions of this paper are as follows:

(1) To address the NP-hard problem of  $L_0$ -norm optimization, an innovative adoption of the  $L_p$ -norm ( $0 < p < 1$ ) is employed to approximate the  $L_0$ -norm in this algorithm. Theoretical proofs demonstrate that when  $p$  is sufficiently small, the solution of the  $L_p$ -norm can be arbitrarily close to the sparse solution of the  $L_0$ -norm, thereby enabling effective sparsification of hidden layer nodes.

(2) The algorithm incorporates the BLinex loss function, which possesses two key characteristics: first, classification calibration, ensuring consistency between model predictions and the true class distribution; second, boundedness, controlling the upper limit of loss through parameters, which can effectively reduce the interference of outliers and noise on the model. Compared with traditional unbounded losses, the BLinex loss performs more stably in noisy data scenarios.

(3) Considering the non-convex and non-smooth nature of the objective function of the model, the algorithm adopts an iterative weighting technique to approximate the  $L_p$  regularizer to a smooth form and combines it with the Adam optimization algorithm for solution. Through double-layer iteration, it is theoretically proven that the iterative process can converge to a local stationary point, significantly improving computational efficiency while ensuring accuracy, making it particularly suitable for large-scale datasets.

This paper is structured as follows: Several classical loss functions are compared in Section 2, where a brief introduction is provided to the BLinex loss function along with its associated models, TELM and Wave-TSVM. The core characteristics of the proposed linear and nonlinear PBLTELM models are elaborated on in Section 3, and complete theorems as well as their proofs regarding the sparsification of hidden layer nodes are presented therein. Detailed experimental results are demonstrated in Section 4 on a two-dimensional artificial dataset, the CMU facial expression image dataset, 12 benchmark datasets from the UCI repository, and 7 benchmark datasets from the relatively large-scale libsvm dataset. Finally, the conclusions of this paper are summarized in Section 5.

## 2. Related work

In order to help readers deeply understand the model proposed in this study, the TELM model, Wave-TSVM model and BLinex loss function are briefly explained in the following sections. In supervised learning

**Table 1**  
Notation.

| Notation                          | Description                                                       | Notation       | Description                                     |
|-----------------------------------|-------------------------------------------------------------------|----------------|-------------------------------------------------|
| $A \in \mathbb{R}^{m_1 \times n}$ | Positive class                                                    | $U, V$         | Hidden layer output matrices                    |
| $B \in \mathbb{R}^{m_2 \times n}$ | Negative class                                                    | $L$            | Number of hidden nodes                          |
| $\beta_1, \beta_2$                | Weight vector                                                     | $\xi, \zeta$   | Slack variable                                  |
| $e_i$                             | Unit vector                                                       | $h(x)$         | Output vector of the hidden layer               |
| $C_1, C_2, C_3, C_4$              | Regularization hyperparameter                                     | $a, b, \gamma$ | Parameter of BLInex loss                        |
| $m_1, m_2$                        | The number of positive and negative samples                       | $e_1, e_2$     | Vectors of ones of appropriate dimensions       |
| $\ x\ _p$                         | $L_p$ -norm( $ x_1 ^p +  x_n ^p$ ) $^{\frac{1}{p}}$ , $0 < p < 1$ | $\ x\ _0$      | Number of non-zero components of $x$            |
| $\ x\ _\infty$                    | Maximum norm                                                      | $x \otimes y$  | The vector whose $i$ -th component is $x_i y_i$ |

tasks, the training set is usually represented as follows:  $D = \{(x_i, y_i)\}_{i=1}^m$ , where  $x_i \in \mathbb{R}^n$  is the input pattern and  $y_i \in \{-1, +1\}$  is the corresponding output. Table 1 summarizes the relevant notations covered in this paper.

## 2.1. TELM

The standard TELM Wan et al. (2017) determines two nonparallel hyperplanes via solving a pair of quadratic programming problems.

$$f_1(x) = h(x)\beta_1 = 0 \quad \text{and} \quad f_2(x) = h(x)\beta_2 = 0.$$

In the random feature space, the TELM model addresses the following two sets of QPPs.

$$\min_{\beta_1, \xi} \frac{1}{2} \|U\beta_1\|_2^2 + C_1 e_2^T \xi, \quad (1)$$

$$\text{s.t.} \quad -V\beta_1 + \xi \geq e_2, \xi \geq 0,$$

and

$$\min_{\beta_2, \zeta} \frac{1}{2} \|V\beta_2\|_2^2 + C_2 e_2^T \zeta, \quad (2)$$

$$\text{s.t.} \quad U\beta_2 + \zeta \geq e_1, \zeta \geq 0,$$

where  $C_1, C_2 > 0$ . Matrices  $U$  and  $V$  represent the hidden layer outputs of data points belonging to the  $+1$  class and the  $-1$  class, respectively. The hidden layer output matrices  $U$  and  $V$  are given by the following equations.

$$U = \begin{bmatrix} u_1 \\ \vdots \\ u_{m_1} \end{bmatrix} = \begin{bmatrix} h_1(x_1) & \cdots & h_L(x_1) \\ \vdots & \ddots & \vdots \\ h_1(x_{m_1}) & \cdots & h_L(x_{m_1}) \end{bmatrix}, \quad (3)$$

$$V = \begin{bmatrix} v_1 \\ \vdots \\ v_{m_2} \end{bmatrix} = \begin{bmatrix} h_1(x_1) & \cdots & h_L(x_1) \\ \vdots & \ddots & \vdots \\ h_1(x_{m_2}) & \cdots & h_L(x_{m_2}) \end{bmatrix}, \quad (4)$$

where  $h(x) = G(\omega_l; b_l; x) = \omega_l \cdot x + b_l$ ,  $l = 1, 2, \dots, L$ , which is defined as the activation function of hidden layer nodes and serves as a key mapping function connecting the input layer and the hidden layer and it is used to convert the input sample  $x$  into the output value of a single hidden layer node after weighting and adjustment by biases. To obtain efficient solutions for the original problems (1) and (2), the corresponding dual problems are derived and presented as follows:

$$\min_{\alpha} \frac{1}{2} \alpha^T V(U^T U + \epsilon I)^{-1} V^T \alpha - e_2^T \alpha, \quad (5)$$

$$\text{s.t.} \quad 0 \leq \alpha \leq C_1 e_2,$$

and

$$\min_{\gamma} \frac{1}{2} \gamma^T U(V^T V + \epsilon I)^{-1} U^T \gamma - e_1^T \gamma, \quad (6)$$

$$\text{s.t.} \quad 0 \leq \gamma \leq C_2 e_1,$$

where  $\alpha$  and  $\gamma$  are vectors of Lagrange multipliers. After the optimal values of  $\alpha$  and  $\gamma$  are obtained by solving Eqs. (5) and (6) respectively, the decision variables  $\beta_1$  and  $\beta_2$  can be utilized to compute.

$$\beta_1 = -(U^T U + \epsilon I)^{-1} V^T \alpha, \quad (7)$$

and

$$\beta_2 = (V^T V + \epsilon I)^{-1} U^T \gamma. \quad (8)$$

For an unseen test data point  $x^* \in \mathbb{R}^n$ , the following decision rule is adopted.

$$f(x^*) = \arg \min_{k=1,2} |h(x^*)\beta_k|.$$

## 2.2. Wave-TSVM

Mushir Akhtar et al. developed the Wave-TSVM Akhtar et al. (2024), which employs the Adam algorithm. The algorithm offers several advantages, such as the introduction of an adaptive learning rate, efficient memory utilization, and rapid convergence during the training process. Given a training dataset  $D$ , the objective of Wave-TSVM is to obtain the positive and negative hyperplanes as follows.

$$f_1(x) = \omega_1^T x + b_1 = 0 \quad \text{and} \quad f_2(x) = \omega_2^T x + b_2 = 0,$$

where  $\omega_1, \omega_2 \in \mathbb{R}^n$  and  $b_1, b_2 \in \mathbb{R}$  denote the weight vectors and bias terms, respectively. The two QPPs that Wave-TSVM needs to solve are

$$\min_{W_1} \frac{1}{2} \|G^T W_1\|_2^2 + C_1 \|W_1\|^2 + C_2 J(W_1), \quad (9)$$

and

$$\min_{W_2} \frac{1}{2} \|H^T W_2\|_2^2 + C_3 \|W_2\|^2 + C_4 J(W_2), \quad (10)$$

where

$$J(W_1) = \sum_{j=1}^{m_2} \frac{1}{\gamma} \left( 1 - \frac{1}{1 + \gamma(1 + H_j^T W_1)^2 \exp\{a(1 + H_j^T W_1)\}} \right), j = 1, \dots, m_2;$$

$$J(W_2) = \sum_{i=1}^{m_1} \frac{1}{\gamma} \left( 1 - \frac{1}{1 - \gamma(1 - G_i^T W_2)^2 \exp\{a(1 - G_i^T W_2)\}} \right), i = 1, \dots, m_1,$$

where  $G_i$  denotes the  $i$  column of matrix  $G$  and  $H_j$  denotes the  $j$  column of matrix  $H$ .  $G = [A, e_1]^T \in \mathbb{R}^{(n+1) \times m_1}$ ,  $H = [B, e_2]^T \in \mathbb{R}^{(n+1) \times m_2}$ ;  $W_1 = [\omega_1^T, b_1]^T \in \mathbb{R}^{(n+1)}$ ,  $W_2 = [\omega_2^T, b_2]^T \in \mathbb{R}^{(n+1)}$ .

The iterative solution procedure with respect to the decision variables is given by

$$W_1^{t+1} = (GG^T + C_1 I)^{-1} \hat{H} q_1^t, \quad (11)$$

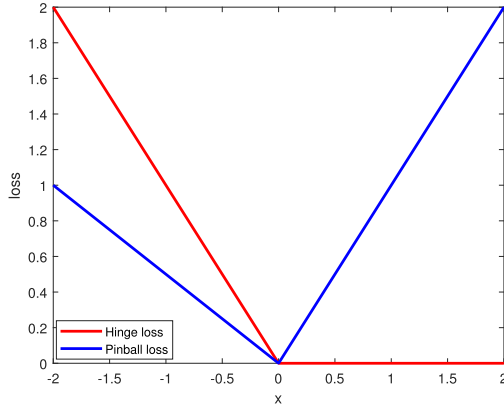
$$W_2^{t+1} = (HH^T + C_3 I)^{-1} \hat{G} q_2^t, \quad (12)$$

where  $\hat{G} = C_4 [G_1, \dots, G_{m_1}] \in \mathbb{R}^{(n+1) \times (m_1)}$ ,  $\hat{H} = C_2 [H_1, \dots, H_{m_2}] \in \mathbb{R}^{(n+1) \times (m_2)}$ ,  $q_1 = [q_{11}, \dots, q_{1m_2}]^T \in \mathbb{R}^{m_2}$ ,  $q_2 = [q_{21}, \dots, q_{2m_1}]^T \in \mathbb{R}^{m_1}$ ;

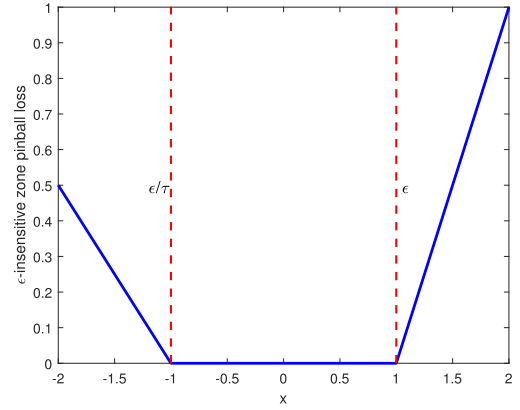
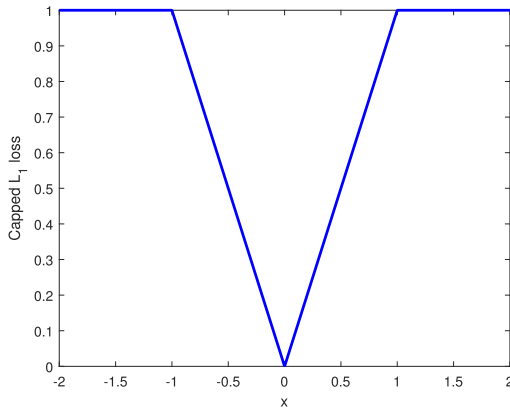
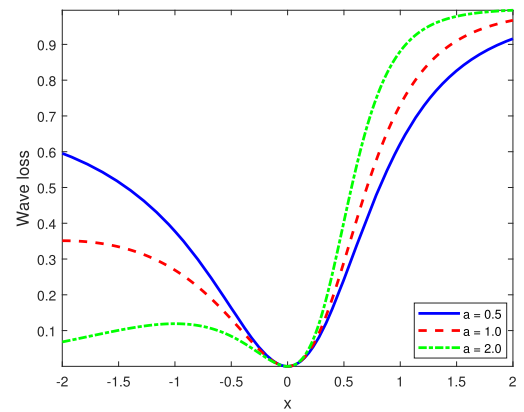
$$q_{1j} = \frac{A_j(aA_j+2)}{(1+\gamma A_j^2 \exp(aA_j))^2}, j = 1, \dots, m_2; q_{2i} = \frac{B_i(aB_i+2)}{(1+\gamma B_i^2 \exp(aB_i))^2}, i = 1, \dots, m_1.$$

The following decision rule is used to determine the class of the unseen sample  $x^* \in \mathbb{R}^n$ .

$$f(x^*) = \arg \min \left( \frac{|\omega_1 x^* + b_1|}{\|\omega_1\|}, \frac{|\omega_2 x^* + b_2|}{\|\omega_2\|} \right).$$



(a) Hinge loss and Pinball loss.

(b)  $\epsilon$ -insensitive zone pinball loss(c) Capped  $L_1$  loss

(d) Wave loss.

**Fig. 1.** Visual illustration of baseline loss functions.

### 2.3. Loss function

The core characteristics of each loss function are systematically analyzed from dimensions such as outlier robustness, noise resistance, convexity, boundedness, smoothness. The specific analysis results are presented in the Table 2 and illustrated in Fig. 1.

From the analysis of relevant properties in Table 2, the Blinex loss is found to possess outlier resistance, noise insensitivity, and the capability to enable sparsification, while also meeting the requirements of boundedness and smoothness. In contrast, all other classical loss functions, including the Wave loss, are unable to cover all these key properties: some lack sparsification capability, while others are deficient in smoothness or boundedness. When compared with the Wave loss, the curve profile of the Blinex loss under different parameter adjustments is more consistent with the smoothness requirement. Thus, the adoption of the Blinex loss can more comprehensively support the performance of TELM variants in terms of model stability, sparsification capability, and optimization efficiency, thereby enhancing their overall performance in relevant tasks.

### 2.4. Blinex loss function

Li et al. developed the BLinex loss function (Tang et al., 2021) and applied it to Bayesian estimation. The Blinex loss function is a bounded

and asymmetric loss function, which is formulated as

$$L(x) = \frac{1}{\gamma} \left[ 1 - \frac{1}{1 + b(e^{ax} - ax - 1)} \right]. \quad (13)$$

Among them,  $a$ ,  $b$ , and  $\gamma$  are parameters. Fig. 2 illustrates the Blinex loss function under different parameter combinations.

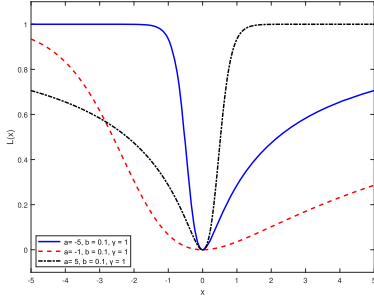
Through visual analysis, the independent regulatory effect of each parameter on the shape of the Blinex loss function can be clearly observed. The sign of parameter  $a$  determines the direction of error penalty, and its absolute value as a scale parameter affects the steepness of the curve. When  $a > 0$ , a smaller value results in a smoother curve, and a larger value makes the curve steeper, while the opposite trend is observed when  $a < 0$ . The parameter  $b$  adjusts the asymmetric property of the loss function, and the smaller its value is, the more significant the asymmetry is. The boundary parameter  $\gamma$  makes use of the boundedness of the loss function to improve the robustness to noise, and can be adjusted to smoothly limit the loss to a preset range. The main advantage of its smoothness is that it facilitates the application of gradient optimization algorithms, ensuring that the gradient is clearly defined throughout the optimization process, so that efficient and reliable optimization methods can be used.

In the following, the theoretical properties of the Blinex loss function are deeply analyzed, and relevant evidence is provided to demonstrate that the Blinex loss function possesses important classification calibration properties. Bartlett et al. (2006) proposed the concept of classifi-

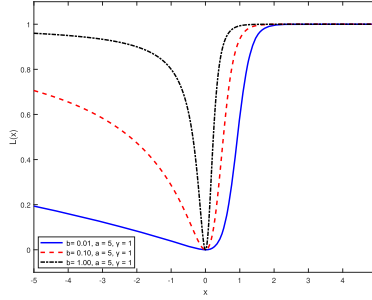
**Table 2**

Comparative analysis of models utilizing different loss functions.

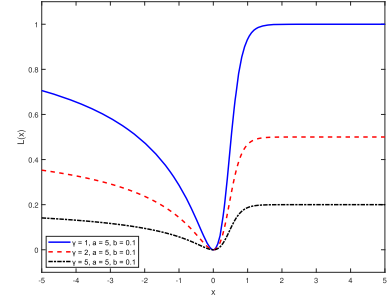
| Method                    | Loss                                      | Robust to outliers | Insensitive to noise | Sparse | Convex | Bounded | Smooth |
|---------------------------|-------------------------------------------|--------------------|----------------------|--------|--------|---------|--------|
| Wan et al. (2017)         | Hinge loss                                | ×                  | ×                    | ×      | ✓      | ×       | ×      |
| Quadri and Tanveer (2024) | Pinball loss                              | ✓                  | ✓                    | ×      | ✓      | ×       | ×      |
| Moosaei and Hladík (2023) | $\epsilon$ -insensitive zone pinball loss | ✓                  | ✓                    | ✓      | ✓      | ×       | ×      |
| Yang et al. (2023)        | Capped $L_1$ loss                         | ✓                  | ✓                    | ×      | ×      | ✓       | ×      |
| Akhtar et al. (2024)      | Wave loss                                 | ✓                  | ✓                    | ×      | ×      | ✓       | ✓      |
| Tang et al. (2021)        | Blinex loss                               | ✓                  | ✓                    | ×      | ×      | ✓       | ✓      |



(a) parameter a



(b) parameter b

(c) parameter  $\gamma$ 

**Fig. 2.** Blinex Loss Function under Different Parameter Conditions. (a) corresponds to different values of  $a$  with  $b = 0.1$  and  $\gamma = 1$  fixed; (b) corresponds to different values of  $b$  with  $a = 5$  and  $\gamma = 1$  fixed; (c) corresponds to different values of  $\gamma$  with  $a = 5$  and  $b = 0.1$  fixed.

cation calibration to evaluate the statistical validity of loss functions in classification learning. Classification calibration ensures that the probability predicted by the model can accurately reflect the actual probability of occurrence of the event. Understanding this property is crucial to further reveal the performance and dynamics of the Blinex loss function in classification tasks.

Consider a dataset  $D = \{x_i, y_i\}_{i=1}^m$  independently drawn from a probability distribution  $P$ , where  $P$  is defined over the joint space of the input space  $X \subseteq \mathbb{R}^n$  and the label space  $Y = \{-1, +1\}$ . Our primary objective is to construct a binary classifier  $C : X \rightarrow Y$  that minimizes the following risk function.

$$R(C) = \int_C P(y \neq C(x)|x) dP_X,$$

where  $P(y|x)$  denotes the conditional probability distribution of label  $y$  given input  $x$ , and  $dP_X$  represents the marginal distribution of input  $x$  based on distribution  $P$ . Since the conditional probability distribution  $P(y|x)$  follows a binomial distribution, its probabilistic properties are fully determined by  $\text{Prob}(y = 1|x)$  and  $\text{Prob}(y = -1|x)$ . To simplify notation, we define  $P(x) = \text{Prob}(y = 1|x)$  and  $1 - P(x) = \text{Prob}(y = -1|x)$ . In essence, the goal of constructing the classifier is to accurately assign labels to input samples and minimize the overall risk based on distribution  $P$ . The Bayesian classifier for  $P(x) \neq \frac{1}{2}$  is defined as

$$f_C(x) = \begin{cases} -1, & P(x) < \frac{1}{2}, \\ 1, & P(x) > \frac{1}{2}. \end{cases}$$

It can be proven that the Bayesian classifier effectively minimizes the classification error. The following expressions mathematically formalize the aforementioned statement.

$$f_C = \arg \min_{C: X \rightarrow Y} R(C).$$

Now, for any given loss function  $L$ , the expected error associated with a classifier  $f : X \rightarrow \mathbb{R}$  is formulated as

$$R_{L,P}(f) = \int_{X \times Y} L(1 - yf(x)) dP.$$

The function  $f_{L,P}$  that minimizes the expected error over all measurable functions is expressed as

$$f_{L,P}(x) = \arg \min_{f(x) \in \mathbb{R}} \int_Y L(1 - yf(x)) dP(y|x), \forall x \in X.$$

Subsequently, a theoretical derivation of the proposed Blinex loss function  $L_{\text{Blinex}}(\cdot)$  is carried out, and [Theorem 1](#) is established, which lays the theoretical foundation for the classification calibration property of the Blinex loss. As the core advantage of this loss function, this property ensures that the result of minimizing the expected risk is consistent with the sign of the Bayesian classifier.

**Theorem 1.** *Blinex loss function  $L_{\text{Blinex}}(\cdot)$  is classification calibrated, i.e.  $f_{L_{\text{Blinex}},P}$  has the same sign as the Bayesian classifier.*

**Proof.** Via straightforward calculations, the following results are obtained.

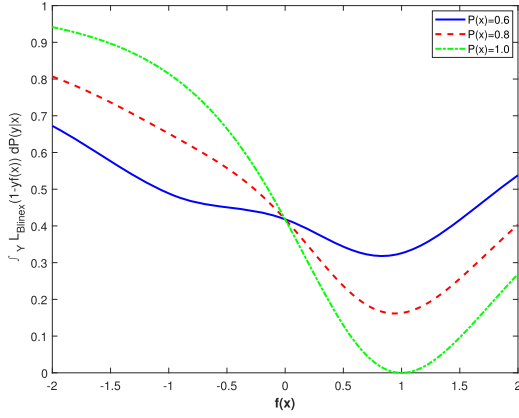
$$\begin{aligned} & \int_Y L_{\text{Blinex}}(1 - yf(x)) dP(y|x) \\ &= L_{\text{Blinex}}(1 - f(x))P(x) + L_{\text{Blinex}}(1 + f(x))(1 - P(x)) \\ &= \frac{1}{\gamma} \left( 1 - \frac{1}{1 + b(e^{a(1-f(x))} - a(1-f(x)) - 1)} \right) P(x) \\ &+ \frac{1}{\gamma} \left( 1 - \frac{1}{1 + b(e^{a(1+f(x))} - a(1+f(x)) - 1)} \right) (1 - P(x)). \end{aligned}$$

In [Fig. 3\(a\)](#) and [3\(b\)](#), the visualizations of  $\int_Y L_{\text{Blinex}}(1 - yf(x)) dP(y|x)$  as a function of  $f(x)$  correspond to the cases where  $P(x) > 1/2$  and  $P(x) < 1/2$ , respectively. The analysis reveals that when  $P(x) > 1/2$ , the minimum of this integral occurs in the region where  $f(x)$  is positive. Conversely, when  $P(x) < 1/2$ , the minimum corresponds to the region where  $f(x)$  is negative.

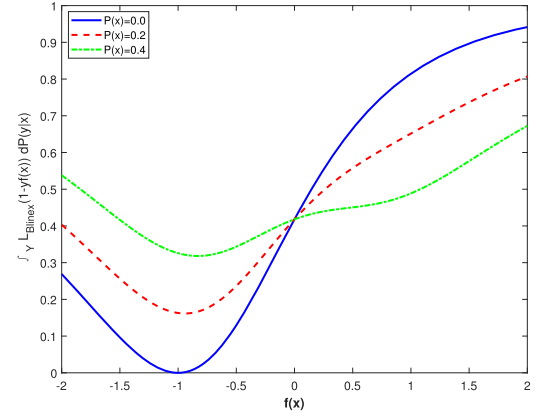
Therefore, from the patterns presented in [Figs. 3\(a\)](#) and [3\(b\)](#), it is clearly evident that the Blinex loss function  $L_{\text{Blinex}}(\cdot)$  exhibits remarkable classification-calibration properties.  $\square$

### 3. $L_p$ -Norm Sparse blinex twin extreme learning machine

Regularization methods are central to the study of sparse optimization problems. By reformulating the original problem into a regularized



(a)



(b)

**Fig. 3.** Illustrate the plot of  $\int_Y L_{\text{Blinex}}(1 - yf(x))dP(y|x)$  over  $f(x)$  with varying of  $P(x)$ . (a)Depict the case for  $P(x) > 1/2$  and (b) illustrate the case for  $P(x) < 1/2$ .

form, it can be transformed into an unconstrained optimization problem. Theoretically, sparse solutions are often obtained through  $L_0$ -norm constraints. However, optimization problems with  $L_0$ -norm in the objective function or constraints have been proven to be NP-hard. To address this challenge, scholars such as Zhang (Zhang et al., 2014) proposed approximating the  $L_0$ -norm with the  $L_p$ -norm ( $0 < p < 1$ ), providing an effective solution approach for such problems. The relevant theoretical proofs will be presented in Theorem 2 of this paper.

Based on the above theoretical foundation, this study integrates the Blinex loss function with the least squares framework of TELM and proposes a novel classification model, namely the  $L_p$ -norm Sparse Blinex Twin Extreme Learning Machine (PBLTELM). This approach exhibits robustness to outliers, insensitivity to noise, and smoothness, making it particularly suitable for solving relatively large-scale problems. In this paper, the Adam optimization technique is employed to solve the PBLTELM problem.

### 3.1. Linear PBLTELM

Similar to TELM, for a given training set  $D$ , the objective of PBLTELM is to obtain two hyperplanes, which are

$$f_1(x) = h(x)\beta_1 = 0 \quad \text{and} \quad f_2(x) = h(x)\beta_2 = 0,$$

where  $\beta_1, \beta_2 \in \mathbb{R}^L$  are the weight vectors. To obtain the above two hyperplanes, the initial problem of PBLTELM is formulated as (linear PBLTELM-1)

$$\min_{\beta_1, \xi} \frac{1}{2} \|U\beta_1\|_2^2 + C_1 \|\beta_1\|_0 + C_2 \sum_{j=1}^{m_2} \xi_j, \quad (14)$$

$$\text{s.t. } \xi_j = \frac{1}{\gamma} \left( 1 - \frac{1}{1 - b(\exp\{a(1 + h(x_j)\beta_1)\} - a(1 + h(x_j)\beta_1) - 1)} \right),$$

(linear PBLTELM-2)

$$\min_{\beta_2, \zeta} \frac{1}{2} \|V\beta_2\|_2^2 + C_3 \|\beta_2\|_0 + C_4 \sum_{i=1}^{m_1} \zeta_i, \quad (15)$$

$$\text{s.t. } \zeta_i = \frac{1}{\gamma} \left( 1 - \frac{1}{1 - b(\exp\{a(1 - h(x_i)\beta_2)\} - a(1 - h(x_i)\beta_2) - 1)} \right),$$

where  $\xi = (\xi_1, \dots, \xi_{m_2}) \in \mathbb{R}^{m_2}$ ,  $\zeta = (\zeta_1, \dots, \zeta_{m_1}) \in \mathbb{R}^{m_1}$ .

The objective function in Eq. (14) contains three components: the first term is used to minimize the distance between the positive hyperplane and positive samples, the second term is a regularization term aimed at enhancing the generalization ability of the model, and the third term is a cumulative penalty term for all negative samples based on the Blinex loss function. Substituting the constraints into the objective

function can transform it into the following unconstrained optimization problem.

$$\min_{\beta_1} \frac{1}{2} \|U\beta_1\|_2^2 + C_1 \|\beta_1\|_0 + C_2 J(\beta_1), \quad (16)$$

and

$$\min_{\beta_2} \frac{1}{2} \|V\beta_2\|_2^2 + C_3 \|\beta_2\|_0 + C_4 J(\beta_2), \quad (17)$$

where

$$J(\beta_1) = \sum_{j=1}^{m_2} \frac{1}{\gamma} \left( 1 - \frac{1}{1 - b(\exp\{a(1 + h(x_j)\beta_1)\} - a(1 + h(x_j)\beta_1) - 1)} \right), j = 1, \dots, m_2;$$

$$J(\beta_2) = \sum_{i=1}^{m_1} \frac{1}{\gamma} \left( 1 - \frac{1}{1 - b(\exp\{a(1 - h(x_i)\beta_2)\} - a(1 - h(x_i)\beta_2) - 1)} \right), i = 1, \dots, m_1.$$

In the objective functions of problems (16) and (17), the  $L_0$  norms  $\|\beta_1\|_0$  and  $\|\beta_2\|_0$  can be approximately replaced by  $L_p$  norms  $\|\beta_1\|_p^p$  and  $\|\beta_2\|_p^p$  (where  $0 < p < 1$ ), thus constructing an  $L_2 - L_p$  minimization problem. This optimization problem exhibits distinct non-smooth and non-convex characteristics. Compared with using the  $L_2$  norm, the introduction of the  $L_p$  norm endows solutions with superior sparsity, manifested as a smaller number of selected features. Therefore, optimization problems (16) and (17) can be transformed into

$$\min_{\beta_1} \frac{1}{2} \|U\beta_1\|_2^2 + C_1 \|\beta_1\|_p^p + C_2 J(\beta_1), \quad (18)$$

and

$$\min_{\beta_2} \frac{1}{2} \|V\beta_2\|_2^2 + C_3 \|\beta_2\|_p^p + C_4 J(\beta_2). \quad (19)$$

In the objective functions of Problems (18) and (19), the second term serves as regularization and controls the sparsity of the classification hyperplane. Below, the equivalence between the unconstrained minimization problems of the  $L_p$ -norm and  $L_0$ -norm is proven. For simplicity, only optimization problem (18) is analyzed (optimization problem (19) exhibits a similar structure). In objective function (18), the  $L_p$  norm  $\|\beta_1\|_p^p$  (where  $0 < p < 1$ ) is used to approximate the  $L_0$  norm  $\|\beta_1\|_0$ , with the core objective of inducing solution sparsity. Inspired by the work of Fung and Mangasarian (2011), it is shown that, under fairly general assumptions, the  $L_0$  norm can provide an arbitrarily precise approximation to the optimization problem. Moreover, as the parameter  $p$  approaches zero, the two problems become fully equivalent both in terms of optimal value and solution. Specifically, it is demonstrated that for a sufficiently small parameter  $p$ , problem (18) and (19) are equivalent to problem (16) and (17), respectively.

**Theorem 2.** Assume the matrix  $U$  is of full column rank. Then, for any  $\varepsilon_0 > 0$ , there exists a  $p \in [0, 1]$  such that the optimal solutions of Eq. (18) and Eq. (16) are at most  $\varepsilon_0$ -far from each other.

**Proof.** Let  $F_0(\beta_1)$  be defined as the objective function of Eq. (16), and  $F_p(\beta_1)$  as that of Eq. (18). It is first noted that the feasible set of Eq. (18) can be restricted to a compact subset. Specifically, there exists a  $\lambda > 0$  such that for each  $p \in [0, 1]$ , every optimal solution of Eq. (18) satisfies  $\|\beta_1\|_\infty \leq \lambda$ , and an analogous result holds for Eq. (16). To verify this, let  $\beta_{1_0} := F_p(0)$ . At this point, all optimal solutions  $\beta_1$  satisfy  $F_p(\beta_1) \leq \beta_{1_0}$ , from which it follows that

$$\|U\beta_1\|_2 \leq \sqrt{2\beta_{1_0}}.$$

Under the given assumption, there exists a submatrix  $\tilde{U}$  of matrix  $U$  such that  $\tilde{U}$  is nonsingular, and analogous arguments apply to other cases. Thus,

$$\|\tilde{U}\beta_1\|_2 \leq \|U\beta_1\|_2 \leq \sqrt{2\beta_{1_0}}.$$

By the properties of matrix norms, the following result is obtained:

$$\|\beta_1\|_2 \leq \|\tilde{U}^{-1}\|_2 \cdot \sqrt{2\beta_{1_0}} = \lambda.$$

Combined with  $\|\beta_1\|_\infty \leq \|\beta_1\|_2$ , i.e. the  $L_\infty$ -norm of any optimal solution  $\beta_1$  of Eq. (18) is bounded. Therefore, It can be seen that the feasible set is a compact set.

Consider the sequence  $p_k = \frac{1}{k}$  ( $k = 2, 3, \dots$ ). Let  $\tilde{\beta}_1^{(k)}$  be the optimal solution to Eq. (18) corresponding to  $p := p_k$ . Via virtue of the aforementioned compactness conclusion, the sequence  $\tilde{\beta}_1^{(k)}$  will converge to a point  $\tilde{\beta}_1$ . It will be shown that this point is an optimal solution to Eq. (16), thereby completing the proof.

Via virtue of the continuity of the corresponding function, for any  $\beta_1$  satisfying  $\|\beta_1\|_\infty \leq \lambda$  and any  $\varepsilon_1, \varepsilon_2, \varepsilon_3 > 0$ , there exists a natural number  $k$  such that

$$\begin{aligned} F_0(\tilde{\beta}_1) &\leq \varepsilon_1 + F_p(\tilde{\beta}_1) \\ &\leq \varepsilon_1 + \varepsilon_2 + F_p(\tilde{\beta}_1^{(k)}) \\ &\leq \varepsilon_1 + \varepsilon_2 + \varepsilon_3 + F_p(\beta_1) \\ &\leq \varepsilon_1 + \varepsilon_2 + \varepsilon_3 + \varepsilon_4 + F_0(\beta_1). \end{aligned}$$

Specifically, the first and fourth inequalities arise from the fact that as  $p$  approaches 0, for a given  $\beta_1$ ,  $\|\beta_1\|_p \rightarrow \|\beta_1\|_0$ ; the second inequality holds because  $\tilde{\beta}_1^{(k)}$  converges to the point  $\tilde{\beta}_1$ ; the third inequality holds because  $\tilde{\beta}_1^{(k)}$  is a minimizer of  $F_p$ . From these inequalities, it follows that  $F_0(\tilde{\beta}_1) \leq F_0(\beta_1)$ , and thus  $\tilde{\beta}_1$  is an optimal solution to Eq. (16).  $\square$

**Remark 1.** In cases where the matrix  $U$  is not of full column rank, numerical stability can be ensured by adding a regularization term  $\epsilon I$  ( $\epsilon > 0$ ) to  $U^T U$ , as implemented in Eq. (7) and (8) of the practical algorithm. Therefore, the implications of the theorem remain relevant for the actual solving process.

The following corollary can be derived.

**Corollary 1.** Assume the matrix  $V$  is of full column rank. Then, for any  $\varepsilon_0 > 0$ , there exists a  $p \in [0, 1]$  such that the optimal solutions of Eq. (19) and Eq. (17) are at most  $\varepsilon_0$ -far from each other.

By constructing a compact feasible set and leveraging sequence convergence and objective function continuity, the equivalence between the  $L_p$  norm ( $0 < p < 1$ ) and  $L_0$  norm as  $p \rightarrow 0$  in PBLTELM has been proved. This conclusion provides a theoretical basis for PBLTELM to achieve sparse hidden layer nodes selection via the  $L_p$  norm. Specifically, when  $p$  is sufficiently small, the solution to the  $L_p$  norm problem can arbitrarily approximate the sparse solution of the  $L_0$  norm problem, thereby effectively sparsification of hidden layer nodes.

**Theorem 3.** Let  $\beta_1^*$  be a local optimal solution to Eq. (18). If  $(\beta_1^*)_i \in (-I_l, I_l)$  for all  $l = 1, \dots, L$ , then  $(\beta_1^*)_i = 0$ , where

$$I_l = \left[ \frac{C_1 p(1-p)}{e_l^T \tilde{U}^T \tilde{U} e_l + C_2 e_l^T Q_1 e_l} \right]^{\frac{1}{2-p}},$$

where  $Q_1 = \frac{a^2 b}{\gamma} \sum_{j=1}^{m_2} h(x_j)^T h(x_j)$  and  $\tilde{U}$  is a submatrix of  $U$ , composed of the columns corresponding to the non-zero components of  $\beta_1^*$ .

**Proof.** Let  $k = \|\beta_1^*\|_0$ . Without loss of generality, let  $\beta_1^* = (\beta_{11}^*, \beta_{12}^*, \dots, \beta_{1k}^*, 0, \dots, 0)^T$ . Let  $Z^* = (\beta_{11}^*, \beta_{12}^*, \dots, \beta_{1k}^*)^T$ . Consider the following optimization problem.

$$\min_Z F_p(Z) = \frac{1}{2} \|\tilde{U} Z\|_2^2 + C_1 \|Z\|_p^p + C_2 J(Z),$$

clearly,  $Z^*$  is a local optimal solution to this problem. Thus, the second-order necessary conditions are satisfied at this point, and the Hessian matrix meets the corresponding conditions.

$$\nabla_Z^2 F_p(Z^*) = \tilde{U}^T \tilde{U} + C_1 p(p-1) \text{diag}(|Z^*|^{p-2}) + C_2 \nabla_{J_1}^2,$$

where  $\nabla_{J_1}^2 = \frac{a^2 b}{\gamma} \sum_{j=1}^{m_2} \frac{A_j \exp(aE_j) - 2b(\exp(aE_j) - 1)^2}{A_j^3} \cdot h(x_j) h(x_j)^T$ ,  $A_j = 1 + b(\exp(aE_j) - aE_j - 1)$ ,  $E_j = 1 + h(x_j)Z$ . It is evident that  $\frac{A_j \exp(aE_j) - 2b(\exp(aE_j) - 1)^2}{A_j^3} \leq 1$  with  $A_j = 1 + b(\exp(aE_j) - aE_j - 1) \geq 1$ ,

so it follows that  $\frac{a^2 b}{\gamma} \cdot \frac{A_j \exp(aE_j) - 2b(\exp(aE_j) - 1)^2}{A_j^3} \cdot h(x_j)^T h(x_j) \leq \frac{a^2 b}{\gamma} \cdot h(x_j)^T h(x_j)$ . Let  $Q_1 = \frac{a^2 b}{\gamma} \sum_{j=1}^{m_2} h(x_j) h(x_j)^T$ . The Hessian matrix is positive semi-definite, so for each  $l = 1, \dots, k$ ,

$$e_l^T \tilde{U}^T \tilde{U} e_l + C_1 p(p-1) |Z^*|^{p-2} + C_2 e_l^T \nabla_{J_1}^2 e_l \geq 0,$$

then,

$$e_l^T \tilde{U}^T \tilde{U} e_l + C_1 p(p-1) |Z^*|^{p-2} + C_2 e_l^T Q_1 e_l \geq 0,$$

it can be derived from this inequality that

$$C_1 p(p-1) \text{diag} |Z^*|^{p-2} \geq -(e_l^T \tilde{U}^T \tilde{U} e_l + C_2 e_l^T Q_1 e_l),$$

$$|Z^*|^{2-p} \geq \frac{C_1 p(p-1)}{e_l^T \tilde{U}^T \tilde{U} e_l + C_2 e_l^T Q_1 e_l},$$

thus,

$$|Z^*| \geq \left[ \frac{C_1 p(p-1)}{e_l^T \tilde{U}^T \tilde{U} e_l + C_2 e_l^T Q_1 e_l} \right]^{\frac{1}{2-p}},$$

therefore, for any local optimal solution  $\beta_1^*$  of Eq. (18), if  $(\beta_1^*)_i \in (-I_l, I_l)$  for  $l = 1, \dots, L$ , then  $\beta_1^* = 0$ .  $\square$

Similar results can also be obtained for Eq. (19). These results have been stated in Theorem 4 without the relevant proof provided, and the proof process is similar to that of Theorem 3.

**Theorem 4.** Let  $\beta_2^*$  be a local optimal solution to Eq. (19). If  $(\beta_2^*)_i \in (-\mathfrak{I}_l, \mathfrak{I}_l)$  for all  $l = 1, \dots, L$ , then  $(\beta_2^*)_i = 0$ , where

$$\mathfrak{I}_l = \left[ \frac{C_3 p(1-p)}{e_l^T \tilde{V}^T \tilde{V} e_l + C_4 e_l^T Q_2 e_l} \right]^{\frac{1}{2-p}},$$

where  $Q_2 = \frac{a^2 b}{\gamma} \sum_{i=1}^{m_1} h(x_i) h(x_i)^T$ , and  $\tilde{V}$  is a submatrix of  $V$ , composed of the columns corresponding to the non-zero components of  $\beta_2^*$ .

Sparse solutions can be obtained and hidden layer nodes selection can be achieved via solving optimization problems (18) and (19). If for some reason fewer than  $N$  hidden layer nodes need to be selected, i.e.,  $\|\beta^*\|_0 < N$  (or  $\|\beta_2^*\|_0 < N$ ) where  $2 \leq N \leq L$ , the following theorem will show that this can be achieved by reasonably selecting the values of parameters  $C_1$  and  $C_3$ .

**Theorem 5.** Let  $\beta_1^*$  be a global optimal solution to problem (18), and

$$C_1 \geq F_p(0) \frac{2-p}{p} N^{\frac{p}{p-2}} \left[ \frac{\mathcal{U} + C_2 \mathfrak{Q}_1}{p(1-p)} \right]^{\frac{p}{2}},$$

where  $\mathcal{U} = \max_{1 \leq l \leq L} e_l^T \tilde{U}^T \tilde{U} e_l$  and  $\mathfrak{Q}_1 = \max_{1 \leq l \leq L} e_l^T Q_1 e_l$ . When  $2 \leq N \leq L$ , it holds that  $\|\beta_1^*\|_0 < N$ .

**Proof.** Suppose  $\beta_1^* \neq 0$  and let  $k = \|\beta_1^*\|_0$ . Without loss of generality,  $\beta_1^*$  can be set as  $(\beta_{11}^*, \beta_{12}^*, \dots, \beta_{1k}^*, 0, \dots, 0)^T$ , and  $Z^*$  can be let as

$(\beta_{11}^*, \beta_{12}^*, \dots, \beta_{1k}^*)^T$ . The following optimization problem can be considered instead of problem (18).

$$\min_Z F_p(Z) = \frac{1}{2} \|\tilde{U}Z\|_2^2 + C_1 \|Z\|_p^p + C_2 J(Z),$$

where  $\tilde{U}$  is a submatrix of  $U$ , composed of the columns corresponding to the non-zero components of  $\beta_1^*$ . Since each element of  $Z^*$  is non-zero, the first-order necessary conditions are satisfied at point  $Z^*$ , and thus it can be obtained that

$$\nabla_Z F_p(Z^*) = \tilde{U}^T \tilde{U} Z^* + C_1 p \text{diag}(|Z^*|^{p-1}) \text{sgn}(Z^*) + C_2 \nabla_{J_1} = 0,$$

where  $\nabla_{J_1} = \sum_{j=1}^{m_2} \frac{ab(\exp(aE_j)-1)}{\gamma A_j^2} \cdot h(x_j)$ ,  $A_j = 1 + b(\exp(aE_j) - aE_j - 1)$ ,  $E_j = 1 + h(x_j)Z$ . Equivalently,

$$\nabla_Z F_p(Z^*) = \tilde{U}^T \tilde{U} Z^* + C_1 p \text{diag}(|Z^*|^{p-2}) Z^* + C_2 \nabla_{J_1} = 0.$$

Since each element of  $Z^*$  is non-zero, at least one of the terms, either  $\tilde{U} Z^*$  or  $\nabla_{J_1}$ , must be zero. Therefore, by substituting  $Z_l^* = \beta_{1l}^*$ , it can be obtained that

$$F_p(Z^*) > C_1 (\|Z^*\|_p^p) = C_1 \sum_{l=1}^k |\beta_{1l}^*|^p.$$

It is known from Theorem 3 that  $\frac{A_j \exp(aE_j) - 2b(\exp(aE_j) - 1)^2}{A_j^3} \leq 1$  with  $A_j = 1 + b(\exp(aE_j) - aE_j - 1) \geq 1$ ,

$$|\beta_{1l}^*| \geq \left[ \frac{C_1 p(1-p)}{e_l^T \tilde{U}^T \tilde{U} e_l + C_2 e_l^T Q_1 e_l} \right]^{\frac{1}{2-p}},$$

thus,

$$F_p(Z^*) > C_1 k \left[ \frac{p(1-p)}{\mathfrak{U} + C_2 \mathfrak{Q}_1} \right]^{\frac{p}{2-p}}.$$

By proving  $F_0(Z^*) \geq F_p(Z^*)$  and using proof by contradiction, it can be shown that  $\|\beta_1^*\|_0 < N$ . If it is derived from the above formula that  $\|\beta_1^*\|_0 = k \geq N$ , then

$$F_p(Z^*) > C_1 N \left[ \frac{p(1-p)}{\mathfrak{U} + C_2 \mathfrak{Q}_1} \right]^{\frac{p}{2-p}}.$$

Similarly, according to the assumption,

$$C_1 \geq F_p(0) \frac{2-p}{p} N^{\frac{p}{p-2}} \left[ \frac{\mathfrak{U} + C_2 \mathfrak{Q}_1}{p(1-p)} \right]^{\frac{p}{2}}.$$

In this case, there will be  $F_p(Z^*) > F_p(0)$ , which contradicts the assumption, and thus the proof is completed.  $\square$

For Eq. (19), a similar conclusion can be derived.

**Theorem 6.** (Upper Bound for  $\beta_1^*$ ). Let  $\mathfrak{U} = \max_{1 \leq l \leq L} e_l^T \tilde{U}^T \tilde{U} e_l$ ,  $\mathfrak{Q}_1 = \sum_{j=1}^{m_2} \frac{ab}{\gamma} \cdot h(x_j)$  and  $\mathfrak{Q}_1 = \max_{1 \leq l \leq L} e_l^T Q_1 e_l$ .  $\tilde{U}$  is a submatrix of  $U$ , composed of the columns corresponding to the non-zero components of  $\beta_1^*$ . Then

$$\|\beta_1^*\| \leq \|(\tilde{U}^T \tilde{U})^{-1}\| \cdot (\|C_1 p \sqrt{k} \left( \frac{p(1-p)}{\mathfrak{U} + C_2 \mathfrak{Q}_1} \right)^{\frac{p-1}{2-p}}\| + \|C_2 \mathfrak{Q}_1\|).$$

**Proof.** Suppose  $\beta_1^* \neq 0$  and let  $k = \|\beta_1^*\|_0$ . Without loss of generality,  $\beta_1^*$  can be set as  $(\beta_{11}^*, \beta_{12}^*, \dots, \beta_{1k}^*, 0, \dots, 0)^T$ , and  $Z^*$  can be let as  $(\beta_{11}^*, \beta_{12}^*, \dots, \beta_{1k}^*)^T$ .

The optimal solution  $\beta_1^*$  must satisfy the first-order necessary condition in Theorem 5. Since each element of  $Z^*$  is non-zero, it can be derived that

$$\tilde{U}^T \tilde{U} Z^* = -C_1 p \text{diag}(|Z^*|^{p-1}) \text{sgn}(Z^*) - C_2 \nabla_{J_1}.$$

For the term  $\nabla_{J_1} = \sum_{j=1}^{m_2} \frac{ab(\exp(aE_j)-1)}{\gamma A_j^2} \cdot h(x_j)$ ,  $\left| \frac{ab(\exp(aE_j)-1)}{\gamma A_j^2} \cdot h(x_j) \right| \leq \left| \frac{ab}{\gamma} \cdot h(x_j) \right|$ , where  $A_j = 1 + b(\exp(aE_j) - aE_j - 1) \geq 1$  and  $E_j = 1 + h(x_j)Z$ . Let  $\mathfrak{Q}_1 = \sum_{j=1}^{m_2} \frac{ab}{\gamma} \cdot h(x_j)$ . Then

$$\|\beta_1^*\| = \|Z^*\|$$

$$\begin{aligned} &= \|(\tilde{U}^T \tilde{U})^{-1} (C_1 p \text{diag}(|Z^*|^{p-1}) \text{sgn}(Z^*) + C_2 \nabla_{J_1})\| \\ &\leq \|(\tilde{U}^T \tilde{U})^{-1}\| \cdot (\|C_1 p \text{diag}(|Z^*|^{p-1})\| + \|C_2 \mathfrak{Q}_1\|) \\ &\leq \|(\tilde{U}^T \tilde{U})^{-1}\| \cdot \left( \left\| C_1 p \sqrt{k} \left( \frac{p(1-p)}{\mathfrak{U} + C_2 \mathfrak{Q}_1} \right)^{\frac{p-1}{2-p}} \right\| + \|C_2 \mathfrak{Q}_1\| \right). \end{aligned}$$

The last inequality above is derived from Theorem 3.  $\square$

For the global optimal solution  $\beta_2^*$  of problem (19), similar boundary conditions can be obtained.

$$\|\beta_2^*\| \leq \|(\tilde{V}^T \tilde{V})^{-1}\| \cdot (\|C_3 p \sqrt{k} \left( \frac{p(1-p)}{\mathfrak{V} + C_4 \mathfrak{Q}_2} \right)^{\frac{p-1}{2-p}}\| + \|C_4 \mathfrak{Q}_2\|).$$

**Theorem 7.** (Lower Bound for  $\beta_1^*$ ). Under the assumptions and notation of Theorem 6, it follows that

$$\|\beta_1^*\| \geq \left( \frac{C_1 p(1-p)}{\mathfrak{U} + C_2 \mathfrak{Q}_1} \right)^{\frac{1}{2-p}}.$$

**Proof.** Theorem 2 can be used to prove this theorem.  $\square$

For the global optimal solution  $\beta_2^*$  of problem (19), similar bounds also exist,

$$\|\beta_2^*\| \geq \left( \frac{C_3 p(1-p)}{\mathfrak{V} + C_4 \mathfrak{Q}_2} \right)^{\frac{1}{2-p}}.$$

In the PBLTELM model, the solution complexity of problems (18) and (19) arises from the non-smooth and non-convex nature of the  $L_p$  norm terms  $\|\beta_1\|_p^p$  and  $\|\beta_2\|_p^p$  in the objective function. Inspired by the study of Zhang and Ye (2017), the original  $L_p$  norms  $\|\beta_1\|_p^p = \sum_{l=1}^L |\beta_{1l}|^p$  and  $\|\beta_2\|_p^p = \sum_{l=1}^L |\beta_{2l}|^p$  are approximated via  $\sum_{l=1}^L (|\beta_{1l}| + \epsilon_0)^p$  and  $\sum_{l=1}^L (|\beta_{2l}| + \epsilon_0)^p$ , where  $\epsilon_0 > 0$  is a tiny constant. The approximation effectively approaches the original  $L_p$  norms as  $\epsilon_0$  tends to zero. Thus, problems (18) and (19) can be solved via the following approximate problems.

$$\min_{\beta_1} \frac{1}{2} \|U \beta_1\|_2^2 + C_1 \sum_{l=1}^L (|\beta_{1l}| + \epsilon_0)^p + C_2 J(\beta_1), \quad (20)$$

and

$$\min_{\beta_2} \frac{1}{2} \|V \beta_2\|_2^2 + C_3 \sum_{l=1}^L (|\beta_{2l}| + \epsilon_0)^p + C_4 J(\beta_2). \quad (21)$$

The objective functions of problems (20) and (21) exhibit non-smooth and non-convex characteristics. To address this challenge, the non-smooth and non-convex terms  $\sum_{l=1}^L (|\beta_{1l}| + \epsilon_0)^p$  and  $\sum_{l=1}^L (|\beta_{2l}| + \epsilon_0)^p$  are approximated with the smooth convex terms  $\|\eta_1 \otimes \beta_1\|_2^2$  and  $\|\eta_2 \otimes \beta_2\|_2^2$ , where  $\eta_1$  and  $\eta_2$  are adaptive adjustment parameters designed to meet the precision approximation requirements. This leads to the construction of the following smoothed optimization problem.

$$\min_{\beta_1} \frac{1}{2} \|U \beta_1\|_2^2 + C_1 \|\eta_1 \otimes \beta_1\|_2^2 + C_2 J(\beta_1), \quad (22)$$

and

$$\min_{\beta_2} \frac{1}{2} \|V \beta_2\|_2^2 + C_3 \|\eta_2 \otimes \beta_2\|_2^2 + C_4 J(\beta_2). \quad (23)$$

In the above optimization problem, the calculation of  $\eta_1$  and  $\eta_2$  constitutes the core solving step. To achieve a more accurate approximation of problems (20) and (21), an iterative optimization strategy is adopted: first, initial values  $\eta_1^{(0)} = (\eta_{11}^{(0)}, \eta_{12}^{(0)}, \dots, \eta_{1L}^{(0)})^T$  and  $\eta_2^{(0)} = (\eta_{21}^{(0)}, \eta_{22}^{(0)}, \dots, \eta_{2L}^{(0)})^T$  are selected. Under the initial conditions of  $\eta_1 = \eta_1^{(0)}$  and  $\eta_2 = \eta_2^{(0)}$ , problems (22) and (23) are solved to obtain  $\beta_1^{(0)}$  and  $\beta_2^{(0)}$ , respectively.

Assume that during the  $k$ -th iteration,  $\beta_1$  and  $\beta_2$  can be estimated by  $\beta_1^{(k)}$  and  $\beta_2^{(k)}$ , respectively. Let  $\beta_1^{(k+1)}$  and  $\beta_2^{(k+1)}$  denote the solutions to the following optimization problems.

$$\min_{\beta_1} \frac{1}{2} \|U \beta_1\|_2^2 + C_1 \|\eta_1^{(k+1)} \otimes \beta_1\|_2^2 + C_2 J(\beta_1), \quad (24)$$

and

$$\min_{\beta_2} \frac{1}{2} \|V\beta_2\|_2^2 + C_3 \|\eta_2^{(k+1)}\|_2^2 + C_4 J(\beta_2), \quad (25)$$

where  $\eta_1^{(k+1)} = (\eta_{11}^{(k+1)}, \eta_{12}^{(k+1)}, \dots, \eta_{1L}^{(k+1)})^T$  and  $\eta_2^{(k+1)} = (\eta_{21}^{(k+1)}, \eta_{22}^{(k+1)}, \dots, \eta_{2L}^{(k+1)})^T$  is the weight vector. It is found that  $\eta_1^{(k+1)}$  and  $\eta_2^{(k+1)}$  ensure that (24) and (25) share the same steepest descent directions as (22) and (23) at the current points  $\beta_1^{(k)}$  and  $\beta_2^{(k)}$ . Therefore,  $\eta_1^{(k+1)}$  and  $\eta_2^{(k+1)}$  satisfy the following equations, respectively.

$$p(|\beta_{1l}^{(k)}| + \epsilon_0)^{p-1} \text{sgn}\beta_{1l}^{(k)} = 2(\eta_1^{(k+1)})^2 \beta_{1l}^{(k)}, \quad (26)$$

and

$$p(|\beta_{2l}^{(k)}| + \epsilon_0)^{p-1} \text{sgn}\beta_{2l}^{(k)} = 2(\eta_2^{(k+1)})^2 \beta_{2l}^{(k)}. \quad (27)$$

Therefore,  $\eta_1^{(k+1)}$  and  $\eta_2^{(k+1)}$  can be rationally chosen. To avoid the occurrence of a zero denominator, a small quantity  $\epsilon_0$  is added, leading to

$$\eta_{1l}^{(k+1)} = \sqrt{\frac{p(|\beta_{1l}^{(k)}| + \epsilon_0)^{p-1}}{2|\beta_{1l}^{(k)}| + \epsilon_0}}, \quad (28)$$

$$\eta_{2l}^{(k+1)} = \sqrt{\frac{p(|\beta_{2l}^{(k)}| + \epsilon_0)^{p-1}}{2|\beta_{2l}^{(k)}| + \epsilon_0}}, \quad (29)$$

(24) and (25) are converted to

$$\min_{\beta_1} \frac{1}{2} \|U\beta_1\|_2^2 + C_1 \|\eta_1'\|_2^2 + C_2 J(\beta_1), \quad (30)$$

and

$$\min_{\beta_2} \frac{1}{2} \|V\beta_2\|_2^2 + C_3 \|\eta_2'\|_2^2 + C_4 J(\beta_2), \quad (31)$$

where  $\eta_1' = \text{diag}(\eta_{11}^{(k+1)}, \eta_{12}^{(k+1)}, \dots, \eta_{1L}^{(k+1)})$ ,  $\eta_2' = \text{diag}(\eta_{21}^{(k+1)}, \eta_{22}^{(k+1)}, \dots, \eta_{2L}^{(k+1)})$ .

Let  $D_1 = \text{diag}(\eta_{11}^{(k+1)}, \eta_{12}^{(k+1)}, \dots, \eta_{1L}^{(k+1)})$ ,  $D_2 = \text{diag}(\eta_{21}^{(k+1)}, \eta_{22}^{(k+1)}, \dots, \eta_{2L}^{(k+1)})$ . Then, (30) and (31) are transformed into

$$\min_{\beta_1} \mathcal{L}(\beta_1) = \frac{1}{2} \|U\beta_1\|_2^2 + C_1 \|D_1\beta_1\|_2^2 + C_2 J(\beta_1), \quad (32)$$

and

$$\min_{\beta_2} \mathcal{L}(\beta_2) = \frac{1}{2} \|V\beta_2\|_2^2 + C_3 \|D_2\beta_2\|_2^2 + C_4 J(\beta_2), \quad (33)$$

where  $J(\beta_1) = \sum_{j=1}^{m_2} (1 - \frac{1}{1-b(\exp\{a(1+h(x_j)\beta_1)\}-a(1+h(x_j)\beta_1)-1)})$ ,  $j = 1, \dots, m_2$ ;  $J(\beta_2) = \sum_{i=1}^{m_1} (1 - \frac{1}{1-b(\exp\{a(1-h(x_i)\beta_2)\}-a(1-h(x_i)\beta_2)-1)})$ ,  $i = 1, \dots, m_1$ . For notational simplicity, define  $E_j = 1 + h(x_j)\beta_1$  and  $F_i = 1 - h(x_i)\beta_2$ .

At each iteration  $t$ ,  $k - th$  samples are first selected. The gradient of Eq. (32), the exponentially decaying average of past gradients (i.e., the first moment), and the squared values of past gradients (i.e., the second moment) are then computed sequentially using Eq. (34), (35), and (36).

$$\nabla \mathcal{L}(\beta_1) = (\hat{U}\hat{U}^T + C_1 D_1 D_1^T)\beta_1 + \hat{V}q_1, \quad (34)$$

$$\mu_{1t} = \kappa_1 \mu_{1t-1} + (1 - \kappa_1) \nabla \mathcal{L}(\beta_{1t}), \quad (35)$$

$$v_{1t} = \kappa_2 v_{1t-1} + (1 - \kappa_2) \nabla \mathcal{L}(\beta_{1t})^2, \quad (36)$$

where  $\hat{U} = C_4 [U_1, \dots, U_{m_1}] \in R^{L \times m_1}$ ,  $\hat{V} = C_2 [V_1, \dots, V_{m_2}] \in R^{L \times m_2}$ ,  $q_1 = [q_{11}, \dots, q_{1m_2}]^T \in R^{m_2}$ ,  $q_{1j} = \frac{a(\exp a E_j - 1)}{\gamma(1+b(\exp a E_j - a E_j - 1))^2}$ ,  $j = 1, \dots, m_2$ , where  $\kappa_1$

and  $\kappa_2$  denote the exponential decay rates for the first and second-moment estimates, respectively, and are typically set to 0.9 and 0.999. Subsequently, the bias-corrected first and second-moment estimates are computed using Eq. (37) and (38), respectively.

$$\hat{\mu}_{1t} = \frac{\mu_{1t}}{1 - \kappa_{1t}}, \quad (37)$$

$$\hat{v}_{1t} = \frac{v_{1t}}{1 - \kappa_{2t}}. \quad (38)$$

Finally, the parameter  $\beta_1$  is updated as follows:

$$\beta_{1t} = \beta_{1t-1} - \alpha \frac{\hat{\mu}_{1t}}{\sqrt{\hat{v}_{1t} + \epsilon_0}}, \quad (39)$$

where  $\alpha$  denotes the learning rate and  $\epsilon_0$  is a small constant to prevent division by zero.

Similarly, the iterative process for solving the optimization of (33) is

$$\nabla \mathcal{L}(\beta_2) = (H H^T + C_3 D_2 D_2^T)\beta_2 + \hat{U}q_2, \quad (40)$$

$$\mu_{2t} = \kappa_1 \mu_{2t-1} + (1 - \kappa_1) \nabla \mathcal{L}(\beta_{2t}), \quad (41)$$

$$v_{2t} = \kappa_2 v_{2t-1} + (1 - \kappa_2) \nabla \mathcal{L}(\beta_{2t})^2, \quad (42)$$

where  $q_2 = [q_{21}, \dots, q_{2m_1}]^T \in R^{m_1}$ ,  $q_{2i} = \frac{a(\exp a F_i - 1)}{\gamma(1+b(\exp a F_i - a F_i - 1))^2}$ ,  $i = 1, \dots, m_1$ .

Then, the bias-corrected first and second moment estimates are computed using Eq. (43) and Eq. (44), respectively.

$$\hat{\mu}_{2t} = \frac{\mu_{2t}}{1 - \kappa_{1t}}, \quad (43)$$

$$\hat{v}_{2t} = \frac{v_{2t}}{1 - \kappa_{2t}}. \quad (44)$$

Finally, the parameter  $\beta_2$  is updated as follows:

$$\beta_{2t} = \beta_{2t-1} - \alpha \frac{\hat{\mu}_{2t}}{\sqrt{\hat{v}_{2t} + \epsilon_0}}, \quad (45)$$

where  $t$  denotes the iteration count, and the iterative process proceeds by solving the iterative Eq. (44) and Eq. (45) until convergence. For classifying an unseen sample  $x^* \in R^n$ , the following decision rule is employed.

$$f(x^*) = \arg \min_{k=1,2} |h(x^*)\beta_k|.$$

The complete algorithm flow chart of the linear PBLTELM is detailed in Fig. 4 which called Algorithm 1.

### 3.2. Nonlinear PBLTELM

The results of the linear PBLTELM are now extended to the nonlinear case, for which a kernel-based surface hyperplane model is introduced.

$$f_1(x) = K(x^T, H^T)\beta_1^\dagger = 0 \quad \text{and} \quad f_2(x) = K(x^T, H^T)\beta_2^\dagger = 0,$$

where  $H = [A, B]^T$ ,  $K(\cdot, \cdot)$  denotes an TELM kernel function encompassing all labeled and unlabeled data. The ELM kernel function can be computed as

$$\begin{aligned} K(x_j, x_i) &= h(x_j) \cdot h(x_i) \\ &= [G(\omega_1; b_1; x_j), \dots, G(\omega_L; b_L; x_j)]^T \\ &\quad \cdot [G(\omega_1; b_1; x_i), \dots, G(\omega_L; b_L; x_i)]. \end{aligned}$$

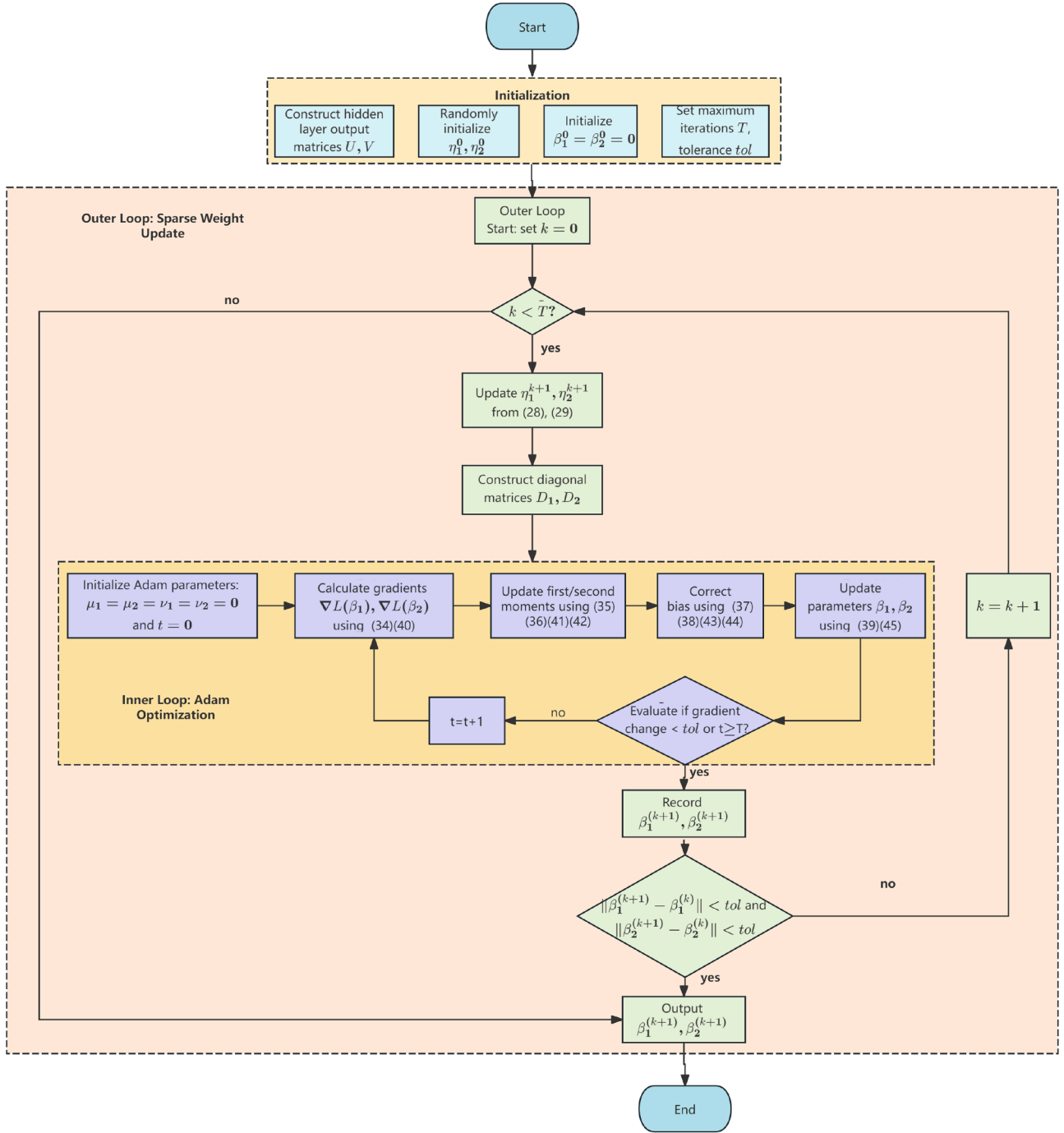


Fig. 4. Algorithm 1.

The nonlinear formulations of optimization problems Eq. (30) and (31) can be expressed as  
(nonlinear PBLTELM-1)

(nonlinear PBLTELM-2)

$$\begin{aligned} \min_{\beta_2^{\dagger}, \zeta^{\dagger}} \quad & \frac{1}{2} \|K(B, H^T)\beta_2^{\dagger}\|_2^2 + C_3 \|\beta_2^{\dagger}\|_p^p + C_4 \sum_{i=1}^{m_1} \zeta_i^{\dagger}, \\ \text{s.t.} \quad & \zeta_i^{\dagger} = \frac{1}{\gamma} \left( 1 - \frac{1}{1 - b(\exp\{a(1 - K(x_j^T, H^T)\beta_2^{\dagger})\} - a(1 - K(x_i^T, H^T)\beta_2^{\dagger}) - 1)} \right). \end{aligned} \quad (47)$$

Problems (46) and (47) can be solved via an iterative approach analogous to that for linear problems. Similarly, at the  $k$ -th iteration,  $\beta_1^{(k)}$  and  $\beta_2^{(k)}$  are used to estimate  $\beta_1^{\dagger}$  and  $\beta_2^{\dagger}$  for problems (46) and (47), respectively. Let  $\beta_1^{\dagger(k+1)}$  and  $\beta_2^{\dagger(k+1)}$  denote the solutions to the following

$$\begin{aligned} \min_{\beta_1^{\dagger}, \zeta^{\dagger}} \quad & \frac{1}{2} \|K(A, H^T)\beta_1^{\dagger}\|_2^2 + C_1 \|\beta_1^{\dagger}\|_p^p + C_2 \sum_{j=1}^{m_2} \zeta_j^{\dagger}, \\ \text{s.t.} \quad & \zeta_j^{\dagger} = \frac{1}{\gamma} \left( 1 - \frac{1}{1 - b(\exp\{a(1 + K(x_j^T, G^T)\beta_1^{\dagger})\} - a(1 + K(x_j^T, H^T)\beta_1^{\dagger}) - 1)} \right), \end{aligned} \quad (46)$$

optimization problems:

$$\min_{\beta_1^\dagger} \frac{1}{2} \|K(A, H^T)\beta_1^\dagger\|_2^2 + C_1 \|\beta_1^{\dagger(k+1)}\|_2^2 + C_2 J(\beta_1^\dagger), \quad (48)$$

and

$$\min_{\beta_2^\dagger} \frac{1}{2} \|K(B, H^T)\beta_2^\dagger\|_2^2 + C_3 \|\beta_2^{\dagger(k+1)}\|_2^2 + C_4 J(\beta_2^\dagger), \quad (49)$$

where  $J(\beta_1^\dagger) = \sum_{j=1}^{m_2} (1 - \frac{1}{1 - b(\exp\{a(1 + K(x_j^T, H^T)\beta_1^\dagger)\} - a(1 + K(x_j^T, H^T)\beta_1^{\dagger(k+1)}) - 1)})$ ,  $j = 1, \dots, m_2$ ;  $J(\beta_2^\dagger) = \sum_{i=1}^{m_1} (1 - \frac{1}{1 - b(\exp\{a(1 - K(x_i^T, H^T)\beta_2^\dagger)\} - a(1 - K(x_i^T, H^T)\beta_2^{\dagger(k+1)}) - 1)})$ ,  $i = 1, \dots, m_1$ .  $\beta_2^{\dagger(k+1)} = (\eta_{11}^{\dagger(k+1)}, \eta_{12}^{\dagger(k+1)}, \dots, \eta_{1L}^{\dagger(k+1)})^T$  and  $\eta_2^{\dagger(k+1)} = (\eta_{21}^{\dagger(k+1)}, \eta_{22}^{\dagger(k+1)}, \dots, \eta_{2L}^{\dagger(k+1)})^T$  are weight vector. For  $l = 1, 2, \dots, L$ ,

$$\eta_{1l}^{\dagger(k+1)} = \sqrt{\frac{p(|\beta_{1l}^{\dagger(k)}| + \epsilon_0)^{p-1}}{2|\beta_{1l}^{\dagger(k)}| + \epsilon_0}}, \quad (50)$$

$$\eta_{2l}^{\dagger(k+1)} = \sqrt{\frac{p(|\beta_{2l}^{\dagger(k)}| + \epsilon_0)^{p-1}}{2|\beta_{2l}^{\dagger(k)}| + \epsilon_0}}. \quad (51)$$

Then (48) and (49) can be converted to

$$\min_{\beta_1^\dagger} \mathcal{L}(\beta_1^\dagger) = \frac{1}{2} \|K(A, H^T)\beta_1^\dagger\|_2^2 + C_1 \|\beta_1^{\dagger(k+1)}\|_2^2 + C_2 J(\beta_1^\dagger), \quad (52)$$

and

$$\min_{\beta_2^\dagger} \mathcal{L}(\beta_2^\dagger) = \frac{1}{2} \|K(B, H^T)\beta_2^\dagger\|_2^2 + C_3 \|\beta_2^{\dagger(k+1)}\|_2^2 + C_4 J(\beta_2^\dagger), \quad (53)$$

where  $D_1^\dagger = \text{diag}(\eta_{11}^{\dagger(k+1)}, \eta_{12}^{\dagger(k+1)}, \dots, \eta_{1L}^{\dagger(k+1)})$ ,  $D_2^\dagger = \text{diag}(\eta_{21}^{\dagger(k+1)}, \eta_{22}^{\dagger(k+1)}, \dots, \eta_{2L}^{\dagger(k+1)})$ . For notational simplicity, denote  $E_j^\dagger = 1 + K(x_j^T, H^T)\beta_1^\dagger$  and  $F_i^\dagger = 1 - K(x_i^T, H^T)\beta_2^\dagger$ , where  $U^\dagger = K(A, H^T)$  and  $V^\dagger = K(B, H^T)$ . The solution to problem (50) follows an approach analogous to that of the linear scenario. The iterative method for Eq. (50) can be derived as

$$\nabla \mathcal{L}(\beta_1^\dagger) = (U^\dagger U^{\dagger T} + C_1 D_1^\dagger D_1^{\dagger T})\tilde{\beta}_1 + \widehat{V}^\dagger q_1^\dagger, \quad (54)$$

$$\mu_{1l}^\dagger = \kappa_1 \mu_{1l-1}^\dagger + (1 - \kappa_1^\dagger) \nabla \mathcal{L}(\beta_{1l}^\dagger), \quad (55)$$

$$v_{1l}^\dagger = \kappa_2^\dagger v_{1l-1}^\dagger + (1 - \kappa_2^\dagger) \nabla \mathcal{L}(\beta_{1l}^\dagger)^2, \quad (56)$$

where  $\widehat{U}^\dagger = C_4 [U_1^\dagger, \dots, U_{m_1}^\dagger] \in R^{L \times m_1}$ ,  $\widehat{V}^\dagger = C_2 [V_1^\dagger, \dots, V_{m_2}^\dagger] \in R^{L \times m_2}$ ,  $q_1^\dagger = [q_{11}^\dagger, \dots, q_{1m_2}^\dagger]^T \in R^{m_2}$ ,  $q_{1j}^\dagger = \frac{a(\exp a E_j^\dagger - 1)}{\gamma(1 + b(\exp a E_j^\dagger - a E_j^\dagger - 1))^2}$ ,  $j = 1, \dots, m_2$ . Then, the bias-corrected first and second moment estimates are computed using Eq. (57) and (58), respectively.

$$\hat{\mu}_{1l}^\dagger = \frac{\mu_{1l}^\dagger}{1 - \kappa_{1l}^\dagger}, \quad (57)$$

$$\hat{v}_{1l}^\dagger = \frac{v_{1l}^\dagger}{1 - \kappa_{2l}^\dagger}. \quad (58)$$

Finally, the parameter  $\beta_1^\dagger$  is updated as follows:

$$\beta_{1l}^\dagger = \beta_{1l-1}^\dagger - \alpha \frac{\hat{\mu}_{1l}^\dagger}{\sqrt{\hat{v}_{1l}^\dagger} + \epsilon_0}. \quad (59)$$

Similarly, the iterative optimization process for (51) is as

$$\nabla \mathcal{L}(\beta_2^\dagger) = (V^\dagger V^{\dagger T} + C_3 D_2^\dagger D_2^{\dagger T})\tilde{\beta}_2 + \widehat{U}^\dagger q_2^\dagger, \quad (60)$$

$$\mu_{2l}^\dagger = \kappa_1^\dagger \mu_{2l-1}^\dagger + (1 - \kappa_1^\dagger) \nabla \mathcal{L}(\beta_{2l}^\dagger), \quad (61)$$

$$v_{2l}^\dagger = \kappa_2^\dagger v_{2l-1}^\dagger + (1 - \kappa_2^\dagger) \nabla \mathcal{L}(\beta_{2l}^\dagger)^2, \quad (62)$$

where  $q_2^\dagger = [q_{21}^\dagger, \dots, q_{2m_1}^\dagger]^T \in R^{m_1}$ ,  $q_{2i}^\dagger = \frac{a(\exp a F_i^\dagger - 1)}{\gamma(1 + b(\exp a F_i^\dagger - a F_i^\dagger - 1))^2}$ ,  $i = 1, \dots, m_1$ . Subsequently, the bias-corrected first and second moment estimates are computed using Eq. (63) and (64), respectively.

$$\hat{\mu}_{2l}^\dagger = \frac{\mu_{2l}^\dagger}{1 - \kappa_{1l}^\dagger}, \quad (63)$$

$$\hat{v}_{2l}^\dagger = \frac{v_{2l}^\dagger}{1 - \kappa_{2l}^\dagger}. \quad (64)$$

Finally, the parameter  $\beta_2^\dagger$  is updated as follows.

$$\beta_{2l}^\dagger = \beta_{2l-1}^\dagger - \alpha \frac{\hat{\mu}_{2l}^\dagger}{\sqrt{\hat{v}_{2l}^\dagger} + \epsilon_0}. \quad (65)$$

All the aforementioned theorems can be adjusted to be applicable to the nonlinear case. Only two of these theorems are presented herein (without proofs), as their proof processes are similar to those in the linear case. The remaining theorems stated in the linear case can also be adjusted in a similar manner to suit the nonlinear case.

**Theorem 8.** Given any local optimal solution  $\beta_1^{\dagger*}$  of Eq. (46). If  $(\beta_1^{\dagger*})_l \in (-I_l^\dagger, I_l^\dagger)$  for all  $l = 1, \dots, L$ , then  $(\beta_1^{\dagger*})_l = 0$ , where

$$I_l^\dagger = \left[ \frac{C_1 p(1-p)}{e_l^T \tilde{U}^{\dagger T} \tilde{U}^\dagger e_l + C_2 e_l^T Q_1^\dagger e_l} \right]^{\frac{1}{2-p}},$$

where  $Q_1^\dagger = \frac{a^2 b}{\gamma} \sum_{j=1}^{m_2} h(x_j) h(x_j)^T$  and  $\tilde{U}^\dagger$  is a submatrix of  $U^\dagger$ , composed of the columns corresponding to the non-zero components of  $\beta_1^{\dagger*}$ .

**Theorem 9.** Let  $\beta_2^{\dagger*}$  be a local optimal solution to Eq. (47). If  $(\beta_2^{\dagger*})_l \in (-\mathfrak{F}_l^\dagger, \mathfrak{F}_l^\dagger)$  for all  $l = 1, \dots, L$ , then  $(\beta_2^{\dagger*})_l = 0$ , where

$$\mathfrak{F}_l^\dagger = \left[ \frac{C_3 p(1-p)}{e_l^T \tilde{V}^{\dagger T} \tilde{V}^\dagger e_l + C_4 e_l^T Q_2^\dagger e_l} \right]^{\frac{1}{2-p}},$$

where  $Q_2^\dagger = \frac{a^2 b}{\gamma} \sum_{i=1}^{m_1} h(x_i) h(x_i)^T$ , and  $\tilde{V}^\dagger$  is a submatrix of  $V^\dagger$ , composed of the columns corresponding to the non-zero components of  $\beta_2^{\dagger*}$ .

For the classification of an unseen sample  $x^* \in \mathbb{R}^n$ , the following decision rule is applied:

$$f(x^*) = \arg \min_{k=1,2} |K(x^{*T}, H^T)\beta_k^\dagger|.$$

### 3.3. Convergence analysis

To prove that PBLTELM implemented via the Adam algorithm achieves convergence, Theorem 10 is first proposed, and the convergence of Algorithm 1 is then proved.

**Theorem 10.** Under the condition that the gradient of the objective function satisfies Lipschitz continuity, the PBLTELM algorithm converges via an iterative process of alternately optimizing the weight vector  $\beta$  and the regularization parameter  $\eta$  to a local stationary point. Here, the sequence  $\{\beta_l\}$  generated by the inner-layer Adam optimization satisfies  $\lim_{l \rightarrow \infty} \mathbb{E}[\|\nabla \mathcal{L}(\beta_l)\|_2^2] = 0$ , and the outer-loop update of  $\eta$  ensures that the overall iteration converges to a stationary point of the objective function.

**Proof.** The inner optimization problem of the algorithm is

$$\min_{\beta} \mathcal{L}(\beta) = \frac{1}{2} \|U\beta\|_2^2 + C_1 \|D\beta\|_2^2 + C_2 J(\beta),$$

where  $J(\beta)$  denotes the loss function term and  $D$  represents a diagonal matrix generated by  $\eta$ . Given that the gradient of  $\mathcal{L}(\beta)$  satisfies the Lipschitz condition.

$$\|\nabla \mathcal{L}(\beta) - \nabla \mathcal{L}(\beta')\| \leq L \|\beta - \beta'\|, \quad \forall \beta, \beta' \in \mathbb{R}^n.$$

Define the first-order moment and second-order moment.

$$\mu_t = \kappa_1 \mu_{t-1} + (1 - \kappa_1) \nabla \mathcal{L}(\beta_t), \quad v_t = \kappa_2 v_{t-1} + (1 - \kappa_2) \|\nabla \mathcal{L}(\beta_t)\|_2^2,$$

after bias correction, it is

$$\hat{\mu}_t = \frac{\mu_t}{1 - \kappa_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \kappa_2^t},$$

the parameter update equation is

$$\beta_t = \beta_{t-1} - \alpha \frac{\hat{\mu}_t}{\sqrt{\hat{v}_t} + \epsilon_0}.$$

Constructing a Lyapunov function  $V_t = \mathcal{L}(\beta_t) + \frac{\alpha^2}{2} \frac{\|\hat{\mu}_t\|_2^2}{\hat{v}_t + \epsilon_0}$ , it follows recursively that

$$\mathbb{E}[V_t] - \mathbb{E}[V_{t-1}] \leq -\frac{\alpha(1 - \kappa_1)}{2G^2} \mathbb{E}[\|\nabla \mathcal{L}(\beta_t)\|_2^2] + \mathcal{O}(\alpha^2),$$

where  $G$  denotes the upper bound of the gradient. When  $\alpha$  is sufficiently small, the right-hand side is non-positive, implying that  $\{\mathbb{E}[V_t]\}$  is monotonically decreasing and bounded below, hence convergent. From  $\sum_{t=1}^{\infty} \mathbb{E}[\|\nabla \mathcal{L}(\beta_t)\|_2^2] < \infty$ , it follows that

$$\lim_{t \rightarrow \infty} \mathbb{E}[\|\nabla \mathcal{L}(\beta_t)\|_2^2] = 0,$$

that is, the sequence  $\{\beta_t\}$  generated by the Adam algorithm converges to a stationary point where the gradient norm is zero.

In the outer layer iteration, the update equation for  $\eta$  is

$$\eta_i^{(k+1)} = \sqrt{\frac{p(|\beta_i^{(k)}| + \epsilon_0)^{p-1}}{2(|\beta_i^{(k)}| + \epsilon_0)}}, \quad p \in (0, 1),$$

this equation ensures that  $\eta^{(k)}$  monotonically approaches the parameter that approximately optimizes the  $L_p$  regularization term, and there exists a constant  $c > 0$  such that

$$\|\eta^{(k+1)} - \eta^{(k)}\| \leq c \|\beta^{(k)} - \beta^{(k-1)}\|.$$

Given  $\eta^{(k)}$ , the inner layer Adam optimization converges to  $\beta^{(k+1)} = \arg \min \mathcal{L}(\beta; \eta^{(k)})$ , satisfying

$$\mathcal{L}(\beta^{(k+1)}; \eta^{(k)}) \leq \mathcal{L}(\beta^{(k)}; \eta^{(k)}) - \frac{\alpha}{2} \|\nabla \mathcal{L}(\beta^{(k)}; \eta^{(k)})\|_2^2.$$

Define the composite objective function  $F(\beta, \eta) = \mathcal{L}(\beta; \eta) + \gamma \|\eta - \eta^*\|$  (where  $\eta^*$  denotes the ideal parameter), then

$$F(\beta^{(k+1)}, \eta^{(k+1)}) \leq F(\beta^{(k)}, \eta^{(k)}) - \delta (\|\nabla \mathcal{L}(\beta^{(k)}; \eta^{(k)})\|_2^2 + \|\eta^{(k+1)} - \eta^{(k)}\|_2^2),$$

where  $\delta > 0$ , indicating that  $\{F(\beta^{(k)}, \eta^{(k)})\}$  is monotonically decreasing and bounded below, hence convergent.

Since the continuity of  $F$  and the compactness of the iterative sequence, let the limit point be  $(\beta^*, \eta^*)$ , which must satisfy

$$\nabla_{\beta} \mathcal{L}(\beta^*; \eta^*) = 0, \quad \nabla_{\eta} F(\beta^*, \eta^*) = 0,$$

i.e.  $(\beta^*, \eta^*)$  is a stationary point of the objective function, thus proving the convergence of the overall iterative process.  $\square$

### 3.4. Complexity analysis

Seven algorithms (PBLTELM, TELM, WLTELM, SPTELM,  $CL_1$ -PTELM, Wave-TSVM, and Pin-GBTSVM) share certain commonalities in the optimization logic and technical pathways for classification tasks, while also exhibiting significant differences due to variations in their core design objectives. In the subsequent sections, a comparative analysis will be conducted with PBLTELM as the core reference, alongside the other six algorithms. Specifically, for TELM, optimization is performed in the ELM feature space, where two small-scale QPPs are solved to determine non-parallel hyperplanes, thereby achieving the optimization of classification tasks. For WLTELM, an optimization framework is constructed using a weighted linear loss function, and model optimization is completed in two steps: in the first step, all training samples are treated equally for initial solution; in the second step, differentiated weights are assigned based on the positions of samples in the feature space for re-optimization, and the entire optimization process can be accomplished merely by solving systems of linear equations. For SPTELM,

an  $\epsilon$ -insensitive zone pinball loss function is introduced, and optimization is realized by solving a pair of QPPs; under the premise of maintaining a computational complexity comparable to that of TELM, both noise insensitivity and sparsity of the model solution are achieved. For  $CL_1$ -PTELM, the basic framework of projection twin extreme learning machine is combined with the capped  $L_1$ -norm distance metric, and the original non-convex optimization problem is transformed into a solvable QPP by means of reweighting technology, so as to enhance the resistance of the model to outliers. For Wave-TSVM, a wave loss function with robustness, noise insensitivity, and smoothness is integrated into the least squares framework of twin support vector machine, and the optimization problem is efficiently handled through a designed iterative algorithm. For Pin-GBTSVM, granular balls are used as input units instead of traditional individual data points to construct the classifier, and a pinball loss function is combined to improve the model's noise insensitivity. For PBLTELM, the BLineX loss function is introduced, the  $L_p$ -norm ( $0 < p < 1$ ) is used to approximate the  $L_0$ -norm for obtaining sparse solutions, and the Adam algorithm is utilized to iteratively optimize the weight vector, so as to solve the non-convex and non-smooth optimization problem and realize high speed and sparsity for the classification of larger-scale datasets. Based on the above sorting of the optimization strategies of each algorithm, the characteristics of the computational complexity of each algorithm will be further analyzed in the following text.

For ease of comparison, the total number of samples is set as  $m$ , where  $m_1$  and  $m_2$  represent the numbers of positive and negative training samples, respectively ( $m = m_1 + m_2$ ).  $T$  denotes the total number of outer iterations, and  $t$  the number of inner iterations;  $L$  is the number of hidden layer nodes, and  $n$  the feature dimension;  $p$  is only used in the Pin-GBTSVM algorithm, referring to the number of generated granular balls. The comparison of the computational complexity of the seven algorithms is shown in the Table 3.

## 4. Numerical experiments

In this section, the performance of PBLTELM was validated on a two-dimensional artificial dataset, as well as UCI datasets and Libsvm datasets. All these methods were implemented in the MATLAB R2024a environment on a personal computer (PC) with a 64-bit Windows XP operating system (OS), and operating at a clock speed of 3.10 GHz.

To validate the effectiveness of PBLTELM, six baseline methods were selected for comparison in this study, including TELM, WLTELM, SPTELM, the  $CL_1$ -PTELM, Wave-TSVM, and Pin-GBTSVM, which were mentioned in Section 3.5. Meanwhile, the performance of PBLTELM and its variants was compared and evaluated to further verify the effectiveness of each component in PBLTELM.

- BLTELM: the sparse module is removed from BLTELM, and the  $L_p$  norm used to represent sparsity is replaced with the  $L_2$  norm.
- PWTELM: the BLineX loss function is replaced with the Wave loss function.

### 4.1. Experiments setting

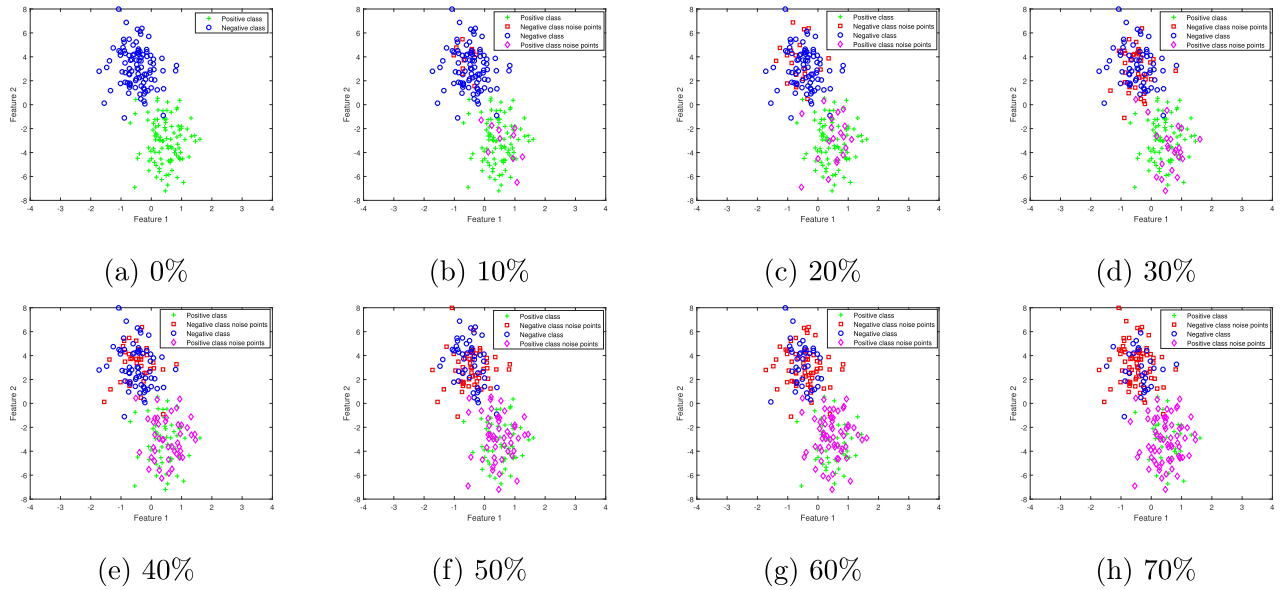
#### 4.1.1. Datasets description

The artificial dataset, CMU dataset, UCI datasets and libsvm large-scale datasets are introduced as follows.

(1) A two-dimensional artificial dataset is constructed, consisting of 200 samples that follow two Gaussian distributions with equal probability. Specifically, positive samples  $x_i$  (where  $y_i = 1$ ,  $i = 1, \dots, 100$ ) follow a Gaussian distribution with the mean vector  $\mu_1 = [0.5, -3]^T$  and the covariance matrix  $\Sigma_1 = \begin{pmatrix} 0.2 & 0 \\ 0 & 3 \end{pmatrix}$ ; negative samples  $x_j$  (where  $y_j = -1$ ,  $j = 1, \dots, 100$ ) follow a Gaussian distribution with the mean vector  $\mu_2 = [-0.5, 3]^T$  and the covariance matrix  $\Sigma_2 = \begin{pmatrix} 0.2 & 0 \\ 0 & 3 \end{pmatrix}$ . Subsequently, label noise is introduced by randomly flipping the labels of samples, with

**Table 3**  
Comparison of Algorithm Computational Complexity.

| Method                                   | Time Complexity                                                                                         | Key Characteristics                                                                                                                                                                                                                      |
|------------------------------------------|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PBLTELM                                  | $O(T \cdot t \cdot (m \cdot n^2 + m \cdot n))$                                                          | Centers on double-layer iterative optimization and comes from weight vector iteration and sample-feature matrix operations.                                                                                                              |
| TELMWan et al. (2017)                    | $O(m^3 + n \cdot m^2)$                                                                                  | Dominated via matrix inversion and linear system solution; the cubic term of total samples corresponds to matrix inversion cost, and the product term of feature dimension and sample square corresponds to linear system solution cost. |
| WLTELMAnand et al. (2023)                | $O(m^2 \cdot n)$                                                                                        | Completes training via two-step linear system solution; avoids high-order iterations or complex matrix operations, with complexity relying on the product of sample square and feature dimension.                                        |
| SPTELMShen and Ma (2019)                 | $O(t \cdot (m \cdot n^2))$                                                                              | Achieves sparse optimization based on $L_p$ norm; computational cost is determined by single iteration count and the product of total samples and feature dimension square.                                                              |
| CL <sub>1</sub> -PTELMYang et al. (2023) | $O(t \cdot (m^3/4 + 2L^3))$                                                                             | Overall efficiency is influenced by iteration count; time cost includes quadratic programming problem solution and matrix-related operations.                                                                                            |
| Wave-TSVMakhtar et al. (2024)            | $\begin{cases} \text{Linear: } O((n+1)^3) \\ \text{Nonlinear: } O(T \cdot (m_1^3 + m_2^3)) \end{cases}$ | Relies on matrix inversion; linear scenario is dominated by feature dimension-related inversion, while nonlinear scenario is affected by iteration count and positive/negative sample sizes.                                             |
| Pin-GBTSVMQuadir and Tanveer (2024)      | $O(p^3/4 + 2mt)$                                                                                        | Computational cost is determined by the number of granular balls, total sample count, and granular generation iteration count.                                                                                                           |



**Fig. 5.** The artificial datasets under the eight noise levels.

eight noise levels ranging from 0% to 70%. The samples with flipped labels retain their original data distribution characteristics while their class labels are incorrectly marked, and these mislabeled points constitute the noise points in the dataset. The artificial datasets under the eight noise levels are shown in the Fig. 5.

(2) The classification task of the image dataset aims to achieve the structured parsing of visual information through an algorithmic model and ultimately reach the goal of automated decision making. For the adopted CMU facial expression image dataset, the side-view expression images of anger and happiness of 20 sample subjects are selected for model validation, and some relevant examples are shown in Fig. 6.

(3) In conducting model validation on UCI datasets, for the tabular data experiment, 12 datasets including Australian, Balance, Cancer, German, Hepar, Ionosphere, Mushroom, Pima, QSAR, Spectf, Vote, and WDBC from the UCI database are selected as research objects. The feature descriptions of the tabular datasets are shown in the following table.

(4) Additionally, to explore the sparsity of PBLTELM and its performance on relatively large-scale datasets, seven relatively large-scale datasets (A1a, A2a, A3a, A7a, A8a, Cod) from the libsvm database are selected as research objects. The feature descriptions of the tabular datasets are shown in Table 4.

#### 4.1.2. Parameters setups

In all experiments of all the algorithms, the Gaussian kernel function (RBF)  $\mathbb{K}(x, x') = \exp\left(-\frac{\|x-x'\|^2}{2\sigma^2}\right)$  was employed to address nonlinear classification problems. Beyond dataset selection and preprocessing, model parameter optimization which is regarded as a core factor affecting performance was equally crucial. The penalty parameter  $C_i$  in each model was utilized to balance training error and model complexity. At the initial stage of the experiments, the number of hidden-layer neurons was set to  $L = 50$ , the tolerance was set to  $\eta = 10^{-5}$ , and the threshold operator was set to  $\epsilon_0 = 10^{-5}$ . Additionally,  $C_1$  was constrained to be equal to  $C_3$ , and  $C_2$  was constrained to be equal to  $C_4$ , with their values selected from the set  $\{10^i \mid i = -5, -4, \dots, 4, 5\}$ . The kernel parameter  $\sigma$  was chosen from  $\{10^i \mid i = -6, -4, \dots, 4, 6\}$ . For TELM variant models other than Wave-TSVM, the parameter  $\tau$  in SPTELM and Pin-GBTSVM was selected from  $\{0.01, 0.1, 0.2, 0.5, 1\}$ ; the parameter  $\epsilon$  in SPTELM and CL<sub>1</sub>-PTELM was chosen from  $\{0, 0.05, 0.1, 0.2, 0.3, 0.5\}$ ; the parameter  $a$  in Wave-TSVM and PBLTELM was selected from  $\{\pm 10^i \mid i = -3, -2, \dots, 2, 3\}$ , the parameter  $b$  was chosen from  $\{10^i \mid i = -3, -2, \dots, 2, 3\}$ , and the bounded parameter  $\gamma$  was fixed at 1. The granular ball purity threshold  $\mathcal{T}$  of Pin-GBTSVM was selected from the set  $\{0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0\}$ , and the parameter  $p$  in PBLTELM

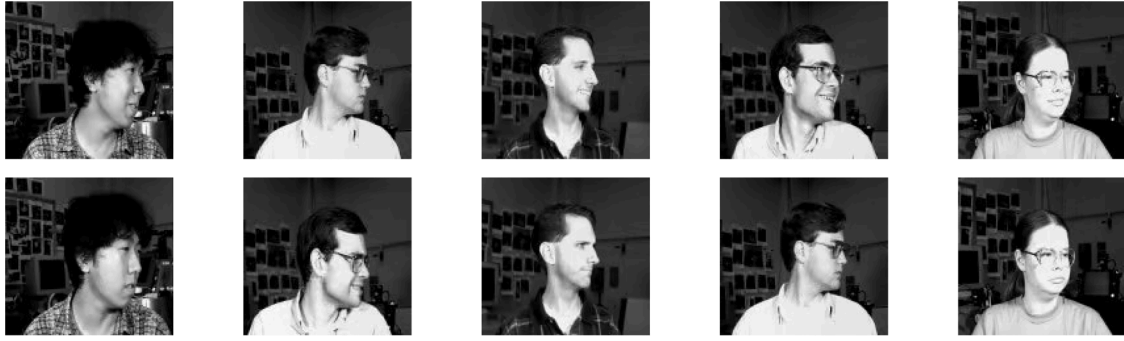


Fig. 6. First row is happy side expression and second row is angry side expression.

**Table 4**  
Characteristics of real-world datasets.

| Datasets   | Samples | Attributes | Source | Datasets   | Samples | Attributes | Source |
|------------|---------|------------|--------|------------|---------|------------|--------|
| Australian | 690     | 14         | UCI    | Balance    | 267     | 4          | UCI    |
| Cancer     | 699     | 9          | UCI    | German     | 1000    | 24         | UCI    |
| Hepat      | 155     | 19         | UCI    | Ionosphere | 306     | 34         | UCI    |
| Mushroom   | 8124    | 112        | UCI    | Pima       | 768     | 8          | UCI    |
| QSAR       | 1055    | 41         | UCI    | Spectf     | 267     | 44         | UCI    |
| Vote       | 435     | 16         | UCI    | WDBC       | 569     | 30         | UCI    |
| A1a        | 30956   | 123        | libsvm | A2a        | 39296   | 123        | libsvm |
| A3a        | 29376   | 123        | libsvm | A5a        | 26147   | 123        | libsvm |
| A7a        | 16461   | 123        | libsvm | A8a        | 9865    | 123        | libsvm |
| Cod        | 59534   | 8          | libsvm |            |         |            |        |

was chosen from the range [0.01 : 0.01 : 0.99]. All experiments are tested using the ten-fold cross-validation method, and the optimal parameter combinations are obtained via grid search.

#### 4.1.3. Evaluation metrics

To evaluate the effectiveness of the proposed PBLTELM with respect to the baseline model, the following defined accuracy metric was employed.

$$ACC = \frac{TP + TN}{TP + FN + TN + FP},$$

where TP and TN are true positive and true negative respectively, FP and FN are false negative and false positive respectively. Meanwhile, to improve the evaluation accuracy of ACC, grid search combined with 10-fold cross-validation was used to optimize experimental parameters. After determining the optimal parameter combination, ten independent repeated experiments were conducted, with the mean value of test results taken as the model performance index. Furthermore, to further evaluate model efficiency, a comparative analysis of training time was performed between the proposed PBLTELM model and baseline models. It should be noted that the reported time only includes the time required to complete model training using optimized parameters. In addition, zero-mean Gaussian noise was added to simulate feature noise, and its variance ratios were set to 0.1 and 0.3, which correspond to the 10% noise and 30% noise referred to in the following context except for the artificial dataset.

## 4.2. Experimental results

### 4.2.1. The results of artificial dataset

In order to evaluate the label noise robustness of the algorithms, experiments were conducted on a synthetic dataset. The results for all seven algorithms are presented in Fig. 7. The figure indicates that under noise-free and low-noise conditions, PBLTELM and the other methods maintain high classification accuracy. As the noise level increases, the performance of all algorithms declines to varying degrees; however, PBLTELM shows relatively stable performance under high noise levels. Specifically, at 60% and 70% noise ratios, PBLTELM achieves higher

classification accuracy than most comparative algorithms, implying that its Blinex loss function and  $L_p$ -norm sparsity strategy contribute to alleviating label noise interference. Overall, the classification performance of PBLTELM in noisy environments corroborates the effectiveness of the anti-noise mechanism outlined.

### 4.2.2. The results of CMU datasets

Next, the performance of seven algorithms on the CMU dataset will be compared. The evaluation metrics employed include the ACC, based on which the classification performance of the seven algorithms will be analyzed.

In the image preprocessing stage, all images are first divided into blocks and uniformly resized to the size of  $30 \times 30$  pixels. At the same time, the gradient direction is divided into 20 intervals, and through the feature fusion technology, 736 dimensional feature vectors are finally extracted from each image. In terms of sample label definition, the images with happy side expressions are classified as positive class, and the images with angry side expressions are classified as negative class. Through the above feature extraction process, the full information collection of 20 portrait images is completed. The classification experimental results of different models on this image dataset are shown in Table 5.

In the emotion classification experiment on 20 portrait images, the classification performance of the PBLTELM model is noteworthy. In the noise-free scenario, the PBLTELM model, along with the TELM, WLTELM, SPTELM, and  $CL_1$ -PTELM models, achieved a classification accuracy of 100%, which is higher than that of the Pin-GBTSVM and Wave-TSVM models. When the noise level increases to 10%, the accuracy of the PBLTELM model is 98.34%, which is higher than that of the TELM, SPTELM,  $CL_1$ -PTELM, Pin-GBTSVM, and Wave-TSVM models and slightly lower than that of the WLTELM model. Under the complex condition where the noise level reaches 30%, the PBLTELM model still maintains an accuracy of 95.96%, outperforming the TELM, SPTELM,  $CL_1$ -PTELM, and Wave-TSVM models and being only slightly inferior to the WLTELM model. The experimental data indicate that the PBLTELM model can maintain relatively stable classification accuracy under different noise environments and exhibits notable advantages in high-noise

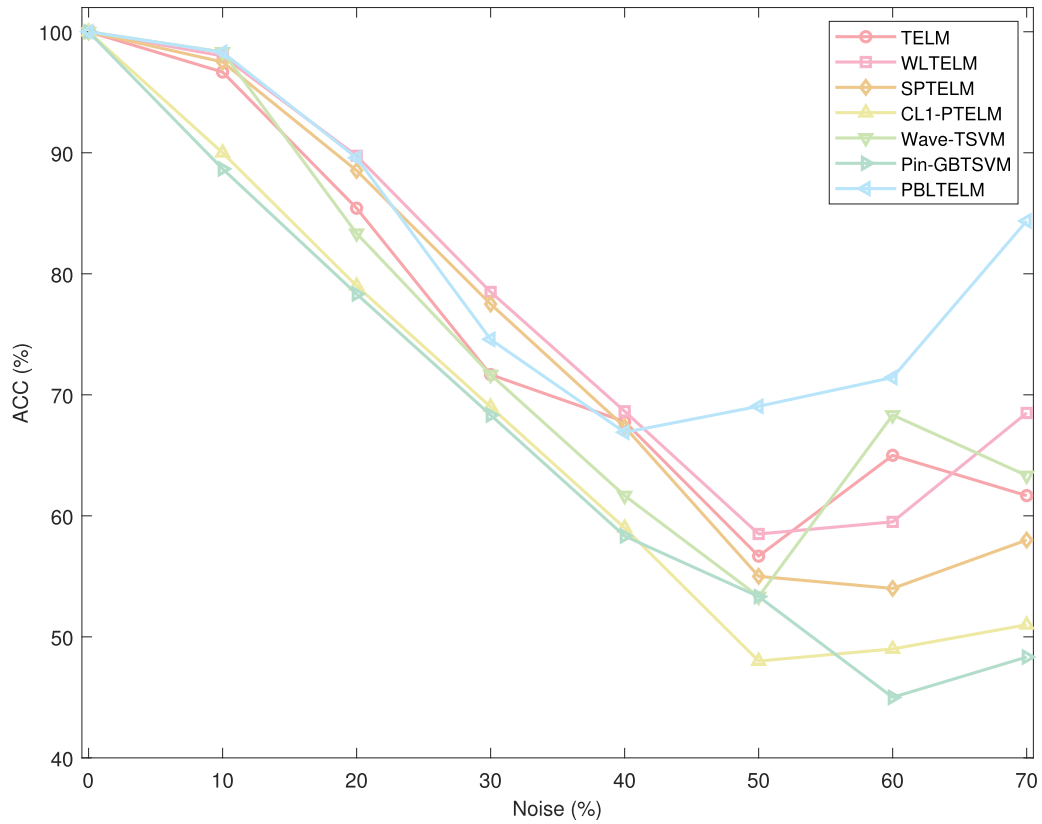


Fig. 7. Line chart of the ACC for seven algorithms on the artificial data.

**Table 5**  
Experimental results on the CMU facial expression dataset.

|       | TELM              | WLTEL             | SPTEL             | CL <sub>1</sub> -PTEL | Wave-TSVM         | Pin-GBTSVM        | PBLTEL            |
|-------|-------------------|-------------------|-------------------|-----------------------|-------------------|-------------------|-------------------|
| noise | ACC(%)<br>Time(s) | ACC(%)<br>Time(s) | ACC(%)<br>Time(s) | ACC(%)<br>Time(s)     | ACC(%)<br>Time(s) | ACC(%)<br>Time(s) | ACC(%)<br>Time(s) |
| 0     | 100<br>0.3987     | 100<br>0.2139     | 100<br>0.0726     | 100<br>0.1273         | 95.11<br>0.0172   | 97.43<br>0.0076   | 100<br>0.0360     |
| 10%   | 97.37<br>0.4123   | 98.25<br>0.2527   | 95.35<br>0.1123   | 91.06<br>0.3215       | 90.07<br>0.0398   | 93.12<br>0.0098   | 98.34<br>0.0567   |
| 30%   | 84.93<br>0.4306   | 93.73<br>0.2791   | 87<br>0.0831      | 83.53<br>0.1598       | 83.77<br>0.0402   | 84.89<br>0.0093   | 95.96<br>0.0762   |

scenarios, making it an image classification model with strong noise resistance.

#### 4.2.3. The results of UCI datasets

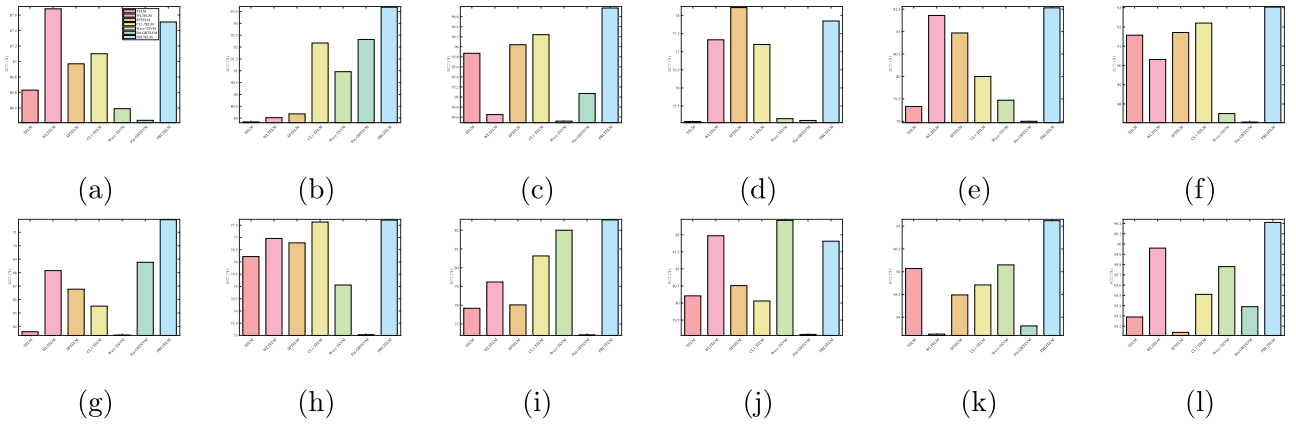
In this section, experiments are conducted based on the dataset in Table 4. The evaluation metrics adopted are the ACC mentioned above, and the running time of the algorithms on the training set.

The UCI dataset is subjected to 10-fold cross-validation, and the performance is measured by the mean value of the test results. The experimental results in Tables 6, 7, and 8 are the optimal outcomes after grid search parameter selection, and the best ACC has been bolded. To analyze the classification performance of these seven algorithms more effectively, their rankings were calculated across different noise environments and datasets, as shown in the Tables 6–8. Additionally, under different noise conditions, bar charts for ACC, violin plots for ACC, and radar charts for the training time of these seven algorithms on the classification training set were plotted, with the corresponding results presented in Figs. 8–12.

The classification performance of PBLTEL on 12 UCI datasets with different noise levels is comprehensively and quantitatively presented through the quantitative data in Tables 6 to 8, the bar charts in Figs. 8

to 10, the violin plot in Fig. 11, and the radar chart of training time in Fig. 12. In the noise-free scenario, the average ACC of PBLTEL is 86.48%, which is higher than that of comparison algorithms including TELM, WLTEL, SPTEL, CL<sub>1</sub>-PTEL, Wave-TSVM, and Pin-GBTSVM. On multiple datasets such as Balance, Cancer, Mushroom, and Vote, the ACC of PBLTEL is optimal, with an average rank of 1.33, which ranks first among all algorithms. The bar chart in Fig. 21 intuitively shows that the ACC of PBLTEL on most datasets is generally higher than that of other models; particularly on the Mushroom dataset, its ACC is higher than that of other algorithms.

When 10% label noise is introduced into the datasets, the average ACC of PBLTEL is 85.56%, which still remains in a leading position, with its average rank further decreasing to 1.17. In the scenario with 30% high noise, the average ACC of PBLTEL is 84.49%, which is still higher than the average level of all comparison algorithms, and its average rank is 1.25, exhibiting a certain level of noise resistance. It can be observed from Table 8 and Fig. 9 that even in a high-noise environment, PBLTEL can still maintain a relatively high classification accuracy on datasets such as Australian, Balance, and QSAR, while its performance is only slightly inferior to that of individual models like CL<sub>1</sub>-PTEL on a few datasets.

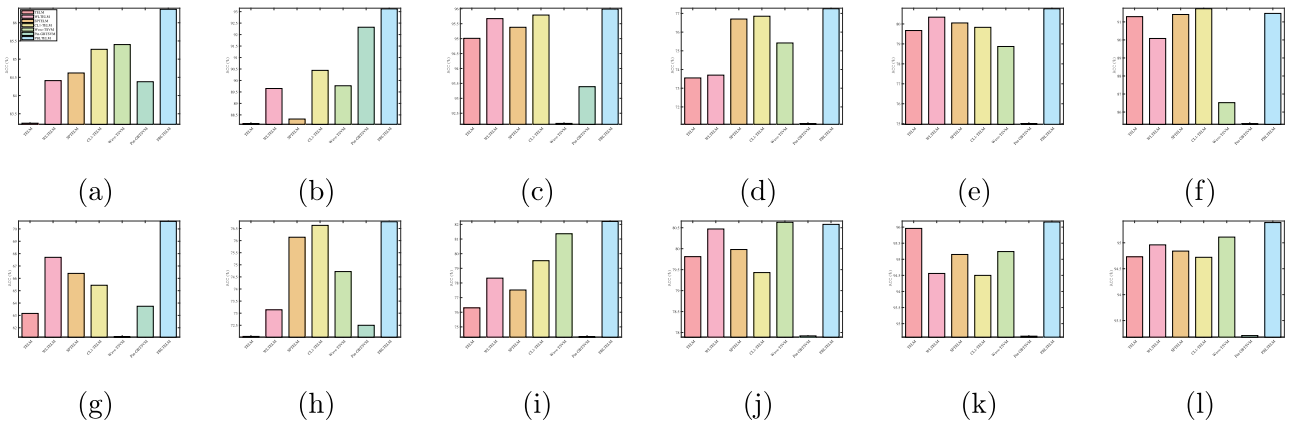


**Fig. 8.** Bar chart of the ACC without noise ((a) to (l) correspond to the Australian, Balance, Cancer, German, Hepat, Ionosphere, Mushroom, Pima, QSAR, Spectf, Vote, and WDBC datasets, respectively).

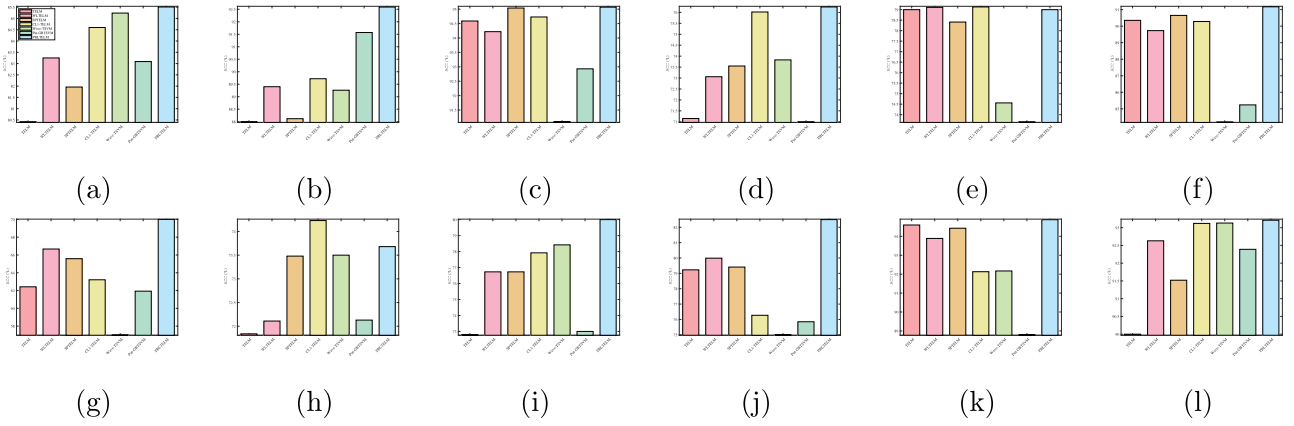
**Table 6**

Performance comparison of seven nonlinear algorithms on UCI datasets without noise.

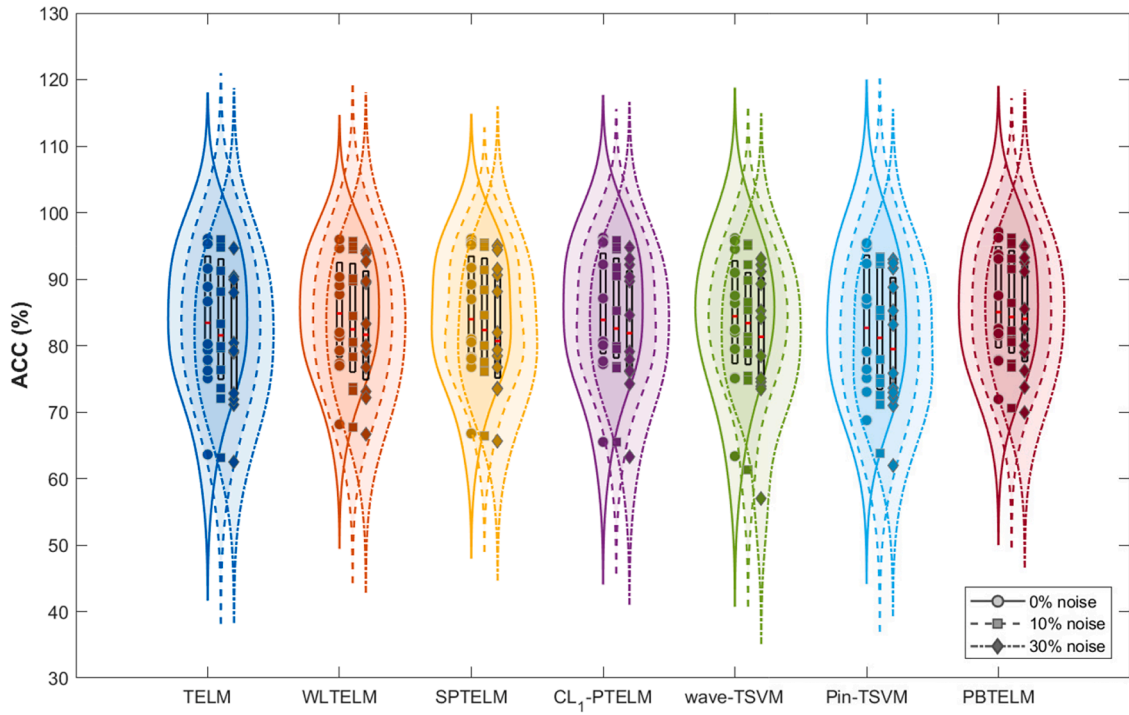
|            | TELM                             | WLTEL                            | SPTEL                            | CL <sub>1</sub> -PTEL            | Wave-TSVM                        | Pin-GBTSVM                       | PBLTEL                           |
|------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|
| Datasets   | ACC + std(%)<br>(Time (s), Rank) | ACC + std(%)<br>(Time (s), Rank) | ACC + std(%)<br>(Time (s), Rank) | ACC + std(%)<br>(Time (s), Rank) | ACC + std(%)<br>(Time (s), Rank) | ACC + std(%)<br>(Time (s), Rank) | ACC + std(%)<br>(Time (s), Rank) |
| Australian | 86.63±2.70<br>(0.1425, 5)        | 87.68±2.42<br>(0.065, 1)         | 86.97±0.55<br>(0.114, 4)         | 87.1±1.5<br>(3.061, 3)           | 86.39±0.1<br>(0.0446, 6)         | 86.24±0.81<br>(0.0503, 7)        | 87.51±0.1<br>(0.1791, 2)         |
| Balance    | 88.84±4.17<br>(0.3427, 7)        | 89.02±4.34<br>(0.0489, 6)        | 89.18±4.03<br>(0.3221, 5)        | 92.17±0.1<br>(1.1293, 3)         | 90.96±0.98<br>(0.0389, 4)        | 92.32±0.0461<br>(0.0508, 2)      | 93.68±0.51<br>(0.1668, 1)        |
| Cancer     | 95.87±0.31<br>(0.5195, 4)        | 94.65±0.46<br>(0.1731, 6)        | 96.04±0.58<br>(0.3325, 3)        | 96.24±0.32<br>(2.0169, 2)        | 94.52±0.50<br>(0.0412, 7)        | 95.07±0.0336<br>(0.0773, 5)      | 96.77±0<br>(0.1932, 1)           |
| German     | 75.07±1.63<br>(1.4077, 7)        | 77.33±1.48<br>(0.4022, 3)        | 78.22±1.6<br>(0.7742, 1)         | 77.2±0.61<br>(6.031, 4)          | 75.15±0.07<br>(0.0707, 5)        | 75.1±0.84<br>(0.0508, 6)         | 77.85±0.07<br>(0.2387, 2)        |
| Hepat      | 79.33±0<br>(0.065, 6)            | 81.36±0.23<br>(0.0325, 2)        | 80.97±0.15<br>(0.0507, 3)        | 80±0.2<br>(1.5472, 4)            | 76.47±0.07<br>(0.0128, 5)        | 79±1.476<br>(0.0578, 7)          | 81.53±0.47<br>(0.1218, 1)        |
| Ionosphere | 91.57±3.72<br>(0.0331, 4)        | 90.31±4.65<br>(0.032, 5)         | 91.71±0.2<br>(0.0261, 3)         | 92.2±0.8<br>(1.437, 2)           | 87.5±0.2<br>(0.0279, 6)          | 87.06±1.125<br>(0.0539, 7)       | 93.03±1.04<br>(0.1488, 1)        |
| Mushroom   | 63.61±2.48<br>(257.571, 6)       | 68.15±1.38<br>(239.1731, 3)      | 66.77±2.44<br>(224.0867, 4)      | 65.51±0.65<br>(309.0852, 5)      | 63.36±1.35<br>(0.4424, 7)        | 68.77±3.86<br>(0.0668, 2)        | 71.93±0.03<br>(3.9993, 1)        |
| Pima       | 76.22±1.06<br>(0.1024, 5)        | 76.96±1.30<br>(0.0375, 3)        | 76.78±0.8<br>(0.1126, 4)         | 77.63±0.13<br>(1.973, 2)         | 75.06±1.02<br>(0.0474, 6)        | 73.03±0.54<br>(0.0692, 7)        | 77.71±0.19<br>(0.183, 1)         |
| QSAR       | 77.84±0.34<br>(1.4116, 6)        | 79.24±1.39<br>(1.2832, 4)        | 78.02±0.3<br>(1.3128, 5)         | 80.63±0.17<br>(7.9693, 3)        | 82±0.07<br>(0.0759, 2)           | 76.42±8.87<br>(0.0566, 7)        | 82.55±0.2<br>(0.2772, 1)         |
| Spectf     | 80.21±0.21<br>(0.31, 5)          | 81.97±0.44<br>(0.11, 2)          | 80.51±0.22<br>(0.1922, 4)        | 80.06±0<br>(1.1142, 6)           | 82.42±1.06<br>(0.0252, 1)        | 79.08±1.415<br>(0.0537, 7)       | 81.81±0.27<br>(0.1316, 3)        |
| Vote       | 96.07±0.17<br>(0.2305, 3)        | 94.63±0.61<br>(0.1938, 7)        | 95.49±0.17<br>(0.2024, 5)        | 95.71±0<br>(1.093, 4)            | 96.15±0.1<br>(0.0314, 2)         | 94.81±0.6<br>(0.0522, 6)         | 97.12±0.17<br>(0.1513, 1)        |
| WDBC       | 95.29±0.25<br>(0.4493, 6)        | 95.96±1.63<br>(0.1498, 2)        | 95.14±0.58<br>(0.3235, 7)        | 95.51±0<br>(1.64, 4)             | 95.78±1.12<br>(0.0416, 3)        | 95.39±1.85<br>(0.0512, 5)        | 96.21±0<br>(0.1665, 1)           |
| avg.ACC    | 83.88                            | 84.77                            | 84.65                            | 85                               | 84.06                            | 83.52                            | 86.48                            |
| avg.Time   | 21.8823                          | 20.1409                          | 18.9875                          | 28.1743                          | 0.0753                           | 0.0576                           | 0.4964                           |
| avg.Rank   | 5.33                             | 3.67                             | 4.00                             | 3.50                             | 4.5                              | 5.67                             | 1.33                             |



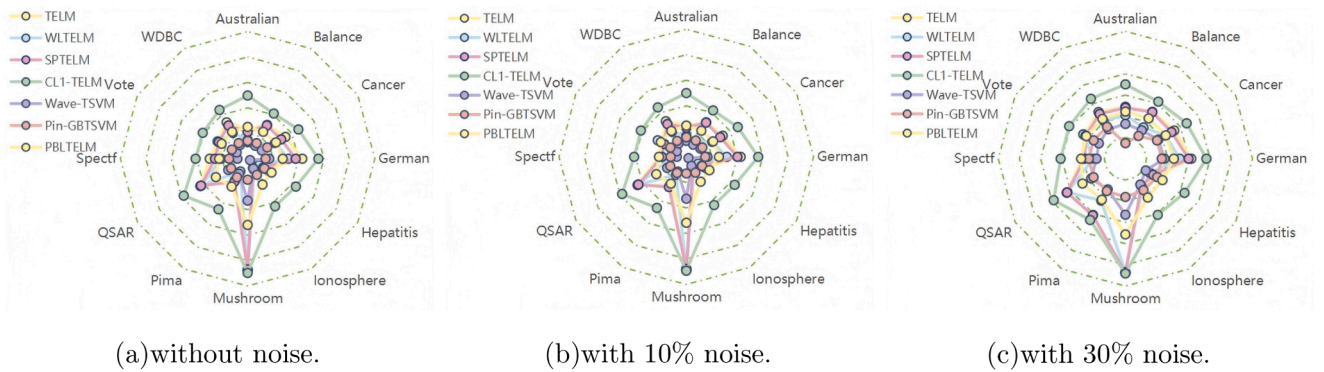
**Fig. 9.** Bar chart of the ACC with 10% noise ((a) to (l) correspond to the Australian, Balance, Cancer, German, Hepat, Ionosphere, Mushroom, Pima, QSAR, Spectf, Vote, and WDBC datasets, respectively).



**Fig. 10.** Bar chart of the ACC with 30% noise ((a) to (l) correspond to the Australian, Balance, Cancer, German, Hepat, Ionosphere, Mushroom, Pima, QSAR, Spectf, Vote, and WDBC datasets, respectively).



**Fig. 11.** Violin plots of ACC for different noise conditions.



**Fig. 12.** Radar plot of the time under different noise conditions.

**Table 7**

Performance comparison of seven nonlinear algorithms on UCI datasets with 10% noise.

|            | TELM                             | WLTELM                           | SPTELM                           | $CL_1$ -PTELM                    | Wave-TSVM                        | Pin-GBTSVM                       | PBLTELM                          |
|------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|
| Datasets   | ACC + std(%)<br>(Time (s), Rank) | ACC + std(%)<br>(Time (s), Rank) | ACC + std(%)<br>(Time (s), Rank) | ACC + std(%)<br>(Time (s), Rank) | ACC + std(%)<br>(Time (s), Rank) | ACC + std(%)<br>(Time (s), Rank) | ACC + std(%)<br>(Time (s), Rank) |
| Australian | 83.24±0.69<br>(0.1662, 7)        | 84.41±0.08<br>(0.0665, 5)        | 84.62±0.55<br>(0.1379, 4)        | 85.27 ± 0.7<br>(3.1597, 3)       | 85.43±0.7<br>(0.0425, 2)         | 84.38 ± 1.44<br>(0.0584, 6)      | 86.37±0.1<br>(0.1671, 1)         |
| Balance    | 88.12±0.13<br>(0.3647, 7)        | 89.65±0.2<br>(0.0521, 5)         | 88.32±0.21<br>(0.3537, 6)        | 90.44±0.06<br>(1.2608, 3)        | 89.77±0.12<br>(0.0353, 4)        | 92.32 ± 0.04<br>(0.055, 2)       | 93.12±0.38<br>(0.1527, 1)        |
| Cancer     | 95.01±0.32<br>(0.4479, 5)        | 95.67±0.44<br>(0.1489, 3)        | 95.38±0.18<br>(0.3449, 4)        | 95.79±0.1<br>(2.1568, 2)         | 92.16±0.05<br>(0.044, 7)         | 93.39 ± 3.36<br>(0.0534, 6)      | 95.99±0.1<br>(0.1636, 1)         |
| German     | 73.55±3.04<br>(1.3598, 6)        | 73.7±2.89<br>(0.3876, 5)         | 76.7±3.01<br>(1.0198, 3)         | 76.85±0.27<br>(6.6703, 2)        | 75.42±1.44<br>(0.0678, 4)        | 71.1 ± 4.63<br>(0.0554, 7)       | 77.25±0.21<br>(0.1915, 1)        |
| Hepat      | 79.67±0.47<br>(0.0689, 5)        | 80.34±1.38<br>(0.0342, 2)        | 80.05±0.65<br>(0.0503, 3)        | 79.83±0.13<br>(1.5688, 4)        | 78.87±0.3<br>(0.0126, 6)         | 75 ± 1.46<br>(0.0521, 7)         | 80.76±1.41<br>(0.1115, 1)        |
| Ionosphere | 91.29±0.5<br>(0.0431, 4)         | 90.08±0.73<br>(0.0391, 5)        | 91.41±0.24<br>(0.0323, 3)        | 91.73 ± 0.8<br>(1.5253, 1)       | 86.52±0.67<br>(0.0254, 6)        | 85.35 ± 0.1169<br>(0.0519, 7)    | 91.47±1.04<br>(0.1337, 2)        |
| Mushroom   | 63.16±2.46<br>(275.7943, 6)      | 67.7±1.02<br>(259.0947, 2)       | 66.4±2.61<br>(245.4569, 3)       | 65.44±3.12<br>(289.584, 4)       | 61.27±3.56<br>(0.4378, 7)        | 63.74 ± 3.85<br>(0.045, 5)       | 70.61±0.18<br>(3.8344, 1)        |
| Pima       | 72.04±1.58<br>(0.1922, 7)        | 73.14±0.26<br>(0.0653, 5)        | 76.14±1.5<br>(0.2326, 3)         | 76.63±0.1<br>(2.0424, 2)         | 74.72±0.31<br>(0.0464, 4)        | 72.50±0.54<br>(0.0535, 6)        | 76.78±0.47<br>(0.1639, 1)        |
| QSAR       | 76.3±1.16<br>(1.6406, 6)         | 78.33±1.23<br>(1.4915, 4)        | 77.52±1.08<br>(1.4929, 5)        | 79.52±0.17<br>(8.1516, 3)        | 81.36±0.55<br>(0.0739, 2)        | 74.33±2.87<br>(0.0533, 7)        | 82.2±0.14<br>(0.229, 1)          |
| Spectf     | 79.81±1.36<br>(0.1161, 5)        | 80.47±1.38<br>(0.0364, 3)        | 79.98±1.1<br>(0.0908, 4)         | 79.43±0<br>(1.1207, 6)           | 80.63±1.62<br>(0.0236, 1)        | 77.92±1.15<br>(0.0415, 7)        | 80.58±1.36<br>(0.1183, 2)        |
| Vote       | 95.96±0<br>(0.1262, 2)           | 94.56±0.05<br>(0.1977, 5)        | 95.15±0.14<br>(0.1022, 4)        | 94.5±0.63<br>(1.1532, 6)         | 95.24±0.01<br>(0.0272, 3)        | 92.61±3.61<br>(0.0531, 7)        | 96.16±0.17<br>(0.143, 1)         |
| WDBC       | 94.73±0.63<br>(0.4075, 5)        | 94.96±0.35<br>(0.1358, 3)        | 94.84±0.33<br>(0.3056, 4)        | 94.72±0.3<br>(1.7312, 6)         | 95.11±0.2<br>(0.0404, 6)         | 93.21±3.75<br>(0.0528, 2)        | 95.39±0.25<br>(0.1495, 1)        |
| avg.ACC    | 82.74                            | 83.58                            | 83.88                            | 84.18                            | 83.04                            | 81.32                            | 85.56                            |
| avg.Time   | 23.3940                          | 21.8125                          | 20.8017                          | 26.6771                          | 0.0731                           | 0.0521                           | 0.4632                           |
| avg.Rank   | 5.42                             | 3.92                             | 3.83                             | 3.50                             | 4.00                             | 6.17                             | 1.17                             |

**Table 8**

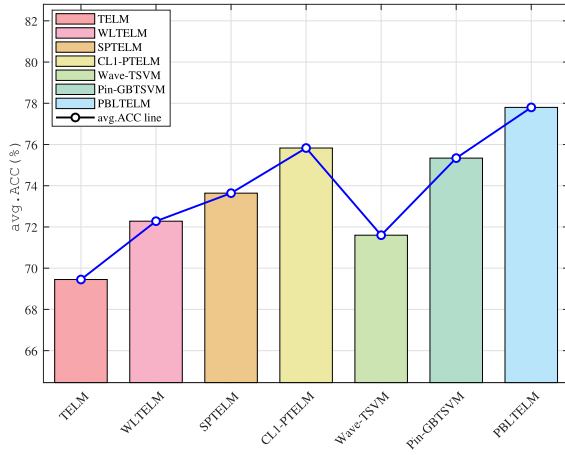
Performance comparison of seven nonlinear algorithms on UCI datasets with 30% noise.

|            | TELM                             | WLTELM                           | SPTELM                           | $CL_1$ -PTELM                    | Wave-TSVM                        | Pin-GBTSVM                       | PBLTELM                          |
|------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|
| Datasets   | ACC + std(%)<br>(Time (s), Rank) | ACC + std(%)<br>(Time (s), Rank) | ACC + std(%)<br>(Time (s), Rank) | ACC + std(%)<br>(Time (s), Rank) | ACC + std(%)<br>(Time (s), Rank) | ACC + std(%)<br>(Time (s), Rank) | ACC + std(%)<br>(Time (s), Rank) |
| Australian | 80.42±0.73<br>(0.2809, 7)        | 83.25±0.15<br>(0.1124, 4)        | 81.96±0.67<br>(0.25, 6)          | 84.6 ± 0.32<br>(3.179, 3)        | 85.24±0.05<br>(0.0436, 2)        | 83.09± 1.51<br>(0.0054, 5)       | 85.51±0.1<br>(0.1656, 1)         |
| Balance    | 88±3.28<br>(0.3513, 7)           | 89.4±0.45<br>(0.0502, 4)         | 88.12±3.2<br>(0.3407, 6)         | 89.72±0.05<br>(1.2683, 3)        | 89.26±0.48<br>(0.0351, 5)        | 91.57 ± 1.85<br>(0.0173, 2)      | 92.59±0.38<br>(0.1557, 1)        |
| Cancer     | 94.59±0.72<br>(0.4581, 4)        | 94.22±0.56<br>(0.1527, 5)        | 95.04±0.04<br>(0.3252, 2)        | 94.73±0.65<br>(2.1722, 3)        | 91.1±1.24<br>(0.0623, 7)         | 92.93 ± 1.03<br>(0.0529, 6)      | 95.07±0.1<br>(0.1582, 1)         |
| German     | 71.15±0.07<br>(1.2811, 6)        | 73.06±1.14<br>(0.366, 5)         | 73.55±2.87<br>(0.9608, 4)        | 76.02±0.16<br>(6.6422, 2)        | 73.83±0.66<br>(0.0687, 3)        | 71 ± 1.57<br>(0.0518, 7)         | 76.25±0.35<br>(0.1932, 1)        |
| Hepat      | 79±1.41<br>(0.043, 3)            | 79.11±1.27<br>(0.0215, 2)        | 78.41±1.26<br>(0.0327, 5)        | 79.13±0.3<br>(1.6628, 1)         | 74.56±0.6<br>(0.0128, 6)         | 73.66 ± 0.14<br>(0.0539, 7)      | 79±0.47<br>(0.1048, 3)           |
| Ionosphere | 90.35±0.62<br>(0.0417, 3)        | 89.73±0.15<br>(0.0379, 5)        | 90.65±0.54<br>(0.0312, 2)        | 90.28±0.75<br>(1.132, 4)         | 84.21±0.24<br>(0.0254, 7)        | 85.24 ± 1.34<br>(0.0511, 6)      | 91.17±0.42<br>(0.1293, 1)        |
| Mushroom   | 62.42±0.09<br>(240.8424, 5)      | 66.67±1.8<br>(219.6394, 2)       | 65.58±0.13<br>(226.3918, 3)      | 63.21±1.06<br>(254.3296, 4)      | 57±0.9<br>(0.4356, 7)            | 61.93 ± 4.04<br>(0.0633, 6)      | 70±0.05<br>(3.6896, 1)           |
| Pima       | 71.84±0.74<br>(1.149, 7)         | 72.11±0.23<br>(0.1916, 6)        | 73.48±0.43<br>(1.2064, 4)        | 74.23±0.1<br>(2.1543, 1)         | 73.5±0.5<br>(0.0467, 3)          | 72.13±3.4<br>(0.0573, 5)         | 73.68±0.74<br>(0.1689, 2)        |
| QSAR       | 72.79±0<br>(1.5852, 7)           | 76.73±1.38<br>(1.4411, 4.5)      | 76.73±0.2<br>(1.5218, 4.5)       | 77.92±0.17<br>(8.1012, 3)        | 78.42±0.13<br>(0.0735, 2)        | 73±0.57<br>(0.0548, 6)           | 80±0.82<br>(0.2186, 1)           |
| Spectf     | 79.23±0.54<br>(0.1066, 4)        | 79.99±0.5<br>(0.0333, 2)         | 79.41±0.4<br>(0.0831, 3)         | 76.27±0<br>(1.1417, 5)           | 75±0.23<br>(0.0231, 7)           | 75.85±4.15<br>(0.0538, 6)        | 82.5±0.27<br>(0.123, 1)          |
| Vote       | 94.6±0.51<br>(0.1733, 2)         | 93.89±0.1<br>(0.1992, 4)         | 94.43±0.25<br>(0.1473, 3)        | 92.13±0.55<br>(1.1608, 6)        | 92.17±0.16<br>(0.0281, 5)        | 88.8±2.64<br>(0.0541, 7)         | 94.88±0.17<br>(0.1358, 1)        |
| WDBC       | 90±4.04<br>(0.3456, 7)           | 92.63±0.44<br>(0.1152, 4)        | 91.52±0.14<br>(0.273, 6)         | 93.12±0.1<br>(1.8218, 3)         | 93.13±0.1<br>(0.0404, 2)         | 92.39±0.85<br>(0.0543, 5)        | 93.21±0<br>(0.1455, 1)           |
| avg.ACC    | 81.20                            | 82.57                            | 82.41                            | 82.61                            | 80.62                            | 80.13                            | 84.49                            |
| avg.Time   | 20.5549                          | 18.53                            | 19.297                           | 23.7305                          | 0.0746                           | 0.0475                           | 0.449                            |
| avg.Rank   | 5.17                             | 3.92                             | 4.00                             | 3.17                             | 4.67                             | 5.67                             | 1.25                             |

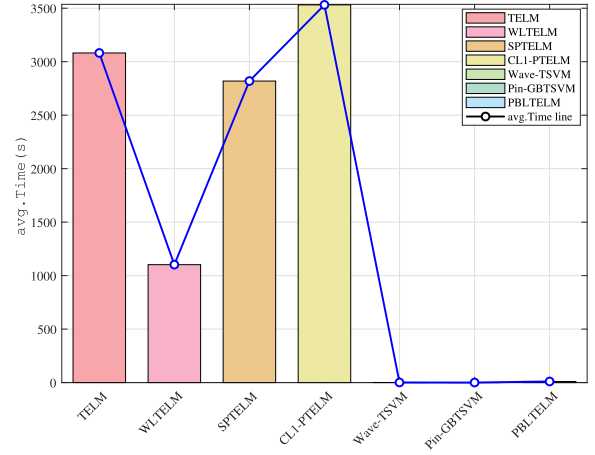
The violin plot in Fig. 11 further reflects the performance stability of PBLTELM: its ACC distribution is more concentrated under different noise conditions, with a smaller fluctuation range than that of other algorithms, which indicates that the model has lower sensitivity to data noise. In terms of training efficiency, when combined with the radar chart in Fig. 12 and the time data in the tables, the training speed of

PBLTELM is lower than that of Wave-TSVM and Pin-GBTSVM, but it is significantly shorter than that of algorithms such as TELM and WLTELM, and falls within a reasonable and efficient range, thus achieving a good balance between classification performance and training efficiency.

Overall, PBLTELM is found to exhibit good classification accuracy and stability on UCI datasets with no noise, 10% noise, and 30% noise,



(a) Average accuracy of seven algorithms.



(b) Time of seven algorithms on training sets.

**Fig. 13.** Classification performance of seven algorithms on relatively large-scale datasets.**Table 9**

Performance comparison of six nonlinear algorithms on libsvm datasets.

|           | TELM      | WLTELM    | SPTELM     | CL <sub>1</sub> -PTELM | Wave-TSVM | Pin-GBTSVM | PBLTELM  |
|-----------|-----------|-----------|------------|------------------------|-----------|------------|----------|
| Datasets  | ACC(%)    | ACC(%)    | ACC(%)     | ACC(%)                 | ACC(%)    | ACC(%)     | ACC(%)   |
|           | Time (s)  | Time (s)  | Time (s)   | Time (s)               | Time (s)  | Time (s)   | Time (s) |
|           | Rank      | Rank      | Rank       | Rank                   | Rank      | Rank       | Rank     |
| A1a       | 70.01     | 68.05     | 74.44      | 75.78                  | 72.13     | 75.53      | 78.3     |
|           | 8506.3536 | 3657.732  | 7060.2734  | 7910.9088              | 1.3556    | 0.5987     | 12.4513  |
|           | 6         | 7         | 4          | 2                      | 5         | 3          | 1        |
| A2a       | 70.01     | 70.02     | 74.77      | 73.28                  | 67.77     | 65.93      | 73.81    |
|           | 8385.2547 | 2861.0489 | 7889.1394  | 9569.2526              | 1.3415    | 0.5917     | 12.1024  |
|           | 5         | 4         | 1          | 3                      | 6         | 7          | 2        |
| A3a       | 72.28     | 75.9      | 75.53      | 74.81                  | 74.03     | 75.5       | 78.22    |
|           | 1167.3857 | 716.6582  | 865.698    | 1056.8731              | 1.337     | 0.4933     | 12.73    |
|           | 7         | 2         | 3          | 5                      | 6         | 4          | 1        |
| A5a       | 74.59     | 75.25     | 75.68      | 78.22                  | 76.07     | 81.99      | 77.23    |
|           | 744.8463  | 366.9954  | 464.9331   | 911.8408               | 1.1473    | 0.4857     | 10.2036  |
|           | 7         | 6         | 5          | 2                      | 4         | 1          | 3        |
| A7a       | 76.67     | 76.17     | 78.66      | 78.97                  | 76.27     | 80         | 80.18    |
|           | 356.5587  | 267.7281  | 259.0399   | 348.1795               | 0.7227    | 0.4739     | 10.2036  |
|           | 5         | 7         | 4          | 3                      | 6         | 2          | 1        |
| A8a       | 74.92     | 82.17     | 74.37      | 83.77                  | 74.96     | 78.32      | 84.98    |
|           | 114.9834  | 79.5488   | 103.1401   | 112.8756               | 0.4707    | 0.1429     | 8.6694   |
|           | 6         | 3         | 7          | 2                      | 5         | 4          | 1        |
| Cod       | 47.69     | 58.42     | 62.06      | 65.98                  | 60        | 70.14      | 71.9     |
|           | 17476.63  | 9911.3812 | 13358.1381 | 18549.698              | 3.1126    | 5.1708     | 27.0047  |
|           | 7         | 6         | 4          | 3                      | 5         | 2          | 1        |
| avg. ACC  | 69.45     | 72.28     | 73.64      | 75.83                  | 71.60     | 75.34      | 77.80    |
| avg. Time | 3081.69   | 1102.53   | 2819.07    | 3531.32                | 0.99      | 0.41       | 10.33    |
| avg. Rank | 6.14      | 5         | 4          | 2.86                   | 5.29      | 3.29       | 1.43     |

and it consistently achieves the top average rank among all comparison algorithms. With a more concentrated ACC distribution, relatively strong noise resistance, and efficient training speed maintained simultaneously, the effectiveness and practicality of PBLTELM in UCI data classification tasks are fully verified.

#### 4.2.4. The results of comparison and discussion for libsvm datasets

Next, the performance advantages and disadvantages of PBLTELM compared to the other six algorithms on relatively large-scale datasets will be explored. The noise resistance of each algorithm on small-scale datasets has been analyzed previously, and here the classification performance of each algorithm in a noise-free scenario will be focused on only. Consistent with the experiments on the UCI dataset, ACC and running time on the training set are still adopted as the evaluation metrics. The experimental data of each algorithm are shown in Table 9.

To compare the performance of each algorithm more clearly, line charts of the average ACC and average training set running time of the seven algorithms are respectively drawn to intuitively present the classification performance of each algorithm on relatively large-scale datasets.

The performance of PBLTELM on 7 noise-free relatively large-scale Libsvm datasets is presented through the quantitative data in Table 9 and the line charts of average ACC and training time in Fig. 13.

It is shown in Table 9 that the average classification accuracy of PBLTELM is higher than that of comparison algorithms such as TELM, WLTELM, and SPTELM, with an average rank of 1.43 that ranks first among all algorithms; its ACC is optimal on datasets including A1a, A3a, A7a, A8a, and Cod. In terms of training efficiency, although the average training time of PBLTELM is lower than that of Wave-TSVM and Pin-GBTSVM, it is much shorter than that of algorithms like TELM and CL<sub>1</sub>-PTELM, falling within a reasonable and efficient range. The line

**Table 10**  
The comparison between the PBLTELM and its variants.

| Datasets   | Method  | Noise 0%                         |         | Noise 10%                        |         | Noise 30%                        |         |
|------------|---------|----------------------------------|---------|----------------------------------|---------|----------------------------------|---------|
|            |         | ACC $\pm$ std                    | Time(s) | ACC $\pm$ std                    | Time(s) | ACC $\pm$ std                    | Time(s) |
| Australian | BLTELM  | 86.62 $\pm$ 0.259                | 0.0884  | 85.26 $\pm$ 1.39                 | 0.094   | 83.37 $\pm$ 3.11                 | 0.1026  |
|            | PWTELM  | 85.18 $\pm$ 3.96                 | 0.2357  | 84.62 $\pm$ 4.05                 | 0.2561  | 84.13 $\pm$ 1.01                 | 0.2542  |
|            | PBLTELM | <b>87.51<math>\pm</math>0.1</b>  | 0.1791  | <b>86.37<math>\pm</math>0.1</b>  | 0.1671  | <b>85.51<math>\pm</math>0.1</b>  | 0.1656  |
| German     | BLTELM  | 75.25 $\pm$ 2.96                 | 0.1437  | 74 $\pm$ 0.76                    | 0.1382  | 72.06 $\pm$ 2.31                 | 0.1684  |
|            | PWTELM  | 75.23 $\pm$ 2.69                 | 0.3362  | 74.46 $\pm$ 1.37                 | 0.3437  | 70 $\pm$ 0.45                    | 0.3507  |
|            | PBLTELM | <b>77.85<math>\pm</math>0.07</b> | 0.2387  | <b>77.25<math>\pm</math>0.21</b> | 0.1915  | <b>76.25<math>\pm</math>0.35</b> | 0.1932  |
| Spectf     | BLTELM  | 81.04 $\pm$ 2.2                  | 0.0459  | <b>80.88<math>\pm</math>2.86</b> | 0.0403  | 80.77 $\pm$ 0.34                 | 0.0319  |
|            | PWTELM  | 80.77 $\pm$ 0                    | 0.1917  | 79.55 $\pm$ 1.68                 | 0.2053  | 76.28 $\pm$ 1.26                 | 0.1917  |
|            | PBLTELM | <b>81.81<math>\pm</math>0.27</b> | 0.1316  | 80.58 $\pm$ 1.36                 | 0.1183  | <b>82.5<math>\pm</math>0.27</b>  | 0.123   |
| WDBC       | BLTELM  | 94.41 $\pm$ 2.5                  | 0.0879  | 92.57 $\pm$ 2.16                 | 0.0857  | 91.2 $\pm$ 1.62                  | 0.0877  |
|            | PWTELM  | <b>96.86<math>\pm</math>0.33</b> | 0.2627  | <b>96.63<math>\pm</math>0.31</b> | 0.2537  | <b>96.5<math>\pm</math>0.21</b>  | 0.2503  |
|            | PBLTELM | 96.21 $\pm$ 0                    | 0.1665  | 95.39 $\pm$ 0.25                 | 0.1495  | 93.21 $\pm$ 0                    | 0.1455  |
| a8a        | BLTELM  | 78.95                            | 1.765   | 75.12                            | 1.7635  | 70.5                             | 1.8426  |
|            | PWTELM  | 80.48                            | 12.1216 | 79.42                            | 13.0278 | 76.37                            | 14.0236 |
|            | PBLTELM | <b>84.98</b>                     | 8.6694  | <b>81.36</b>                     | 8.7873  | <b>78.06</b>                     | 9.7247  |
| cod-rna    | BLTELM  | 65.33                            | 7.1892  | 63.29                            | 9.5657  | 56.78                            | 11.2259 |
|            | PWTELM  | 67.23                            | 34.0263 | 63.78                            | 67.0237 | 60.04                            | 40.0277 |
|            | PBLTELM | <b>71.9</b>                      | 27.0047 | <b>67.42</b>                     | 35.7526 | <b>65.33</b>                     | 27.9834 |

charts in Fig. 13 intuitively confirm this trend: the average ACC curve of PBLTELM is consistently higher than that of other algorithms, while its training time curve is significantly lower than that of most comparison algorithms.

#### 4.3. Ablation study

A systematic ablation study was performed to verify the importance of each component in the PBLTELM framework, confirming their practical significance rather than random combination, with details of removed components in Section 4.1. Six datasets of varying scales and characteristics (Australian, German, Spectf, WDBC, A8a, Cod) were selected, covering 0%, 10%, and 30% noise scenarios. Taking ACC and running time as core metrics, systematic analysis was conducted using data from Table 10 and Figs. 14–18 to clarify the role of each component in performance improvement, where Fig. 17 depicts ACC distribution and robustness, and Fig. 18 quantifies temporal stability.

The contribution of the sparsity module was verified by comparing PBLTELM with BLTELM. In terms of classification accuracy, the ACC of PBLTELM was generally observed to be superior to that of BLTELM across most scenarios. As illustrated in Fig. 17, the ACC distribution of PBLTELM exhibited higher kurtosis and a more concentrated range. For instance, on the German dataset with 30% noise, an ACC of 76.25% was recorded for PBLTELM, which was higher than the 72.06% achieved by BLTELM. The dispersion of the ACC distribution for PBLTELM was also noted to be lower than that of BLTELM as noise levels increased, indicating that the sparsity module enhances model robustness against noise by eliminating redundant features. Regarding computational efficiency, the running time of BLTELM was generally lower than that of PBLTELM. The radar chart in Fig. 18 showed a more compact profile for BLTELM across datasets, reflecting the additional computational overhead introduced by the sparse optimization. However, the temporal fluctuation of PBLTELM across different noise scenarios was smaller, suggesting that the module contributes to computational stability at an acceptable cost.

The role of the Blinex loss function, which provides classification calibration and boundedness, was examined by comparing PBLTELM with PWTELM. A superior ACC performance for PBLTELM was observed on most datasets. In Fig. 17, the ACC distribution of PBLTELM was characterized by shorter tails and fewer outliers. This was particularly evident on the Spectf dataset with 30% noise, where an ACC of 82.50% was achieved by PBLTELM, compared to 76.28% for PWTELM. The ACC distribution of PWTELM was flatter and exhibited greater fluctuation,

indicating that the Blinex loss function helps optimize predictive consistency. In terms of computational stability, the running time of PWTELM, as shown in the radar chart in Fig. 18, demonstrated noticeable variability across noise scenarios, with significant shifts in its profile. In contrast, the temporal distribution of PBLTELM remained largely consistent, with highly overlapping contours, suggesting that the smoothness of the Blinex loss is better suited to the Adam optimizer, reducing computational instability caused by the characteristics of the loss function.

In summary, quantitative data and visualization results from ablation experiments collectively verify that each core component of the PBLTELM framework exhibits irreplaceable functional value. Differences in the contribution of each component are consistent with the characteristics of task scenarios, and such differences are not only reflected in quantitative data but also intuitively manifested through visualization results, thereby effectively mitigating potential misjudgment of trends that might arise from relying solely on numerical analysis. These results fully verify that the framework design of PBLTELM is not a random combination of components but a systematic optimization tailored to task requirements.

#### 4.4. Convergence analysis

Section 3.3 delves into the model proposed through theoretical convergence analysis. The convergence of PBLTELM is verified through experiments in this section. The convergence curves of the algorithm, including the inner-layer and outer-layer convergence curves, are shown in Fig. 16.

In Fig. 16, the images in the first row are the inner-layer convergence curves of the algorithm on several datasets of different scales, where the vertical axis represents the gradients of  $\beta_1$  and  $\beta_2$ ; the images in the second row are the corresponding outer-layer convergence curves, with the vertical axis representing  $\|\beta_i^{(k+1)} - \beta_i^{(k)}\|$ . Fig. 16 shows that in all four classification tasks, the objective function decreases rapidly within the first 50 iterations and stabilizes afterward, which reflects the efficient optimization performance of the PBLTELM model. In addition, the convergence curves of all classification tasks eventually tend to be stable, indicating no significant fluctuations during this period.

#### 4.5. Parameter analysis

This section analyzes the sensitivity of key parameters  $p$ ,  $C_i$ ,  $a$ , and  $b$  in the PBLTELM algorithm. The experiment selects six datasets with

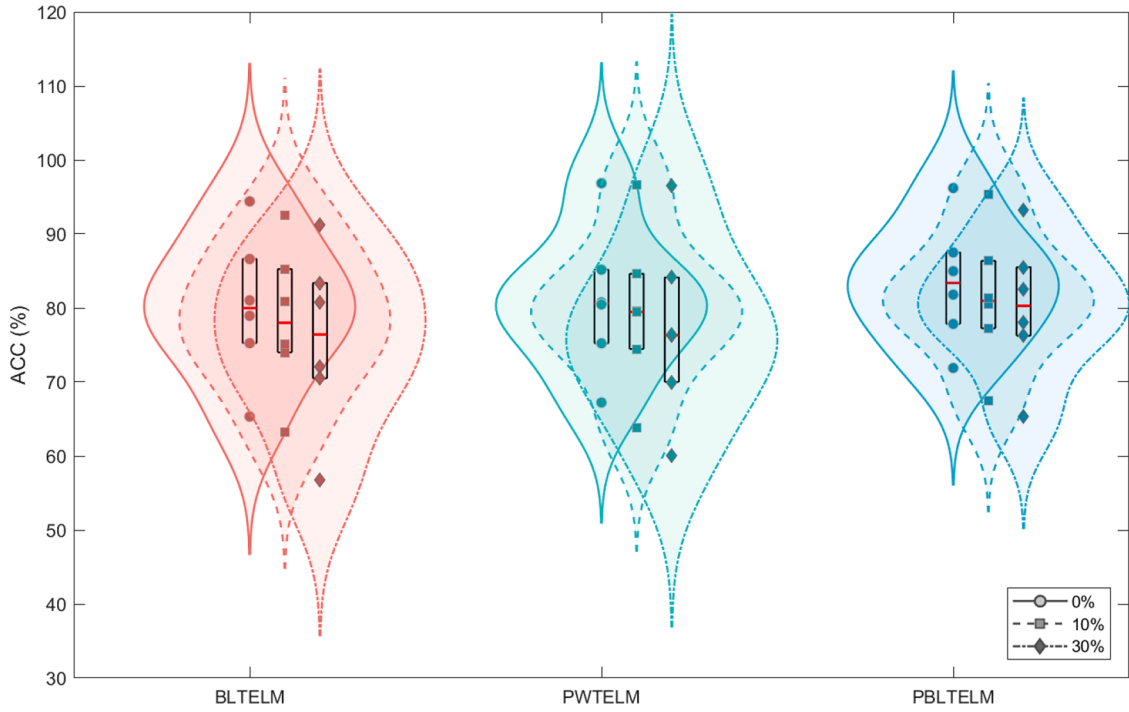


Fig. 14. Violin plots of ACC of PBLTELM and its variants for different noise conditions.

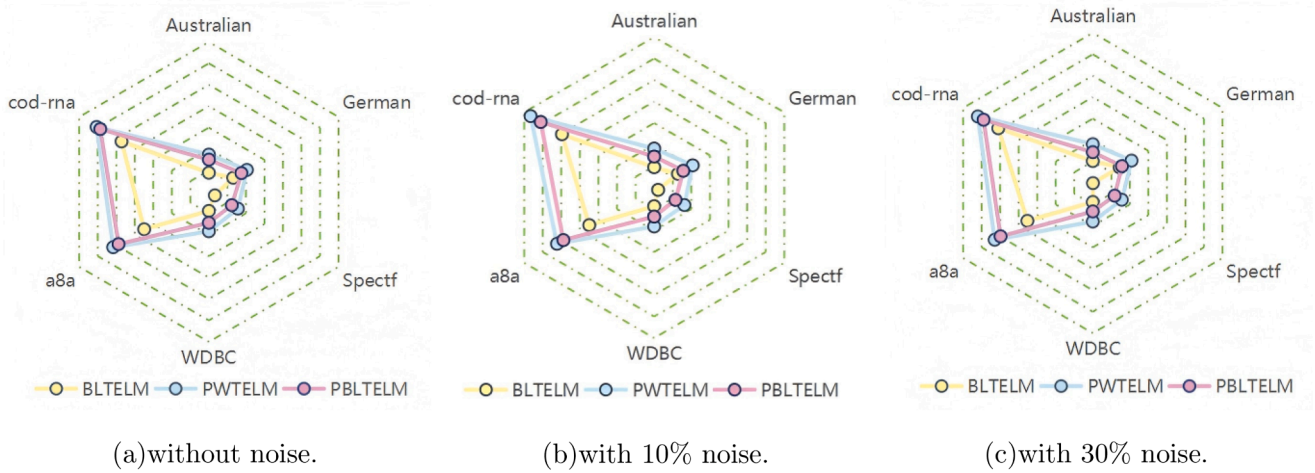


Fig. 15. Radar plot of the time of PBLTELM and its variants under different noise conditions.

different scales and noise levels, including Australian, German, Ionosphere, Spectf, Vote, and WDBC, and uses the control variable method for research. The results are presented in Figs. 19–20.

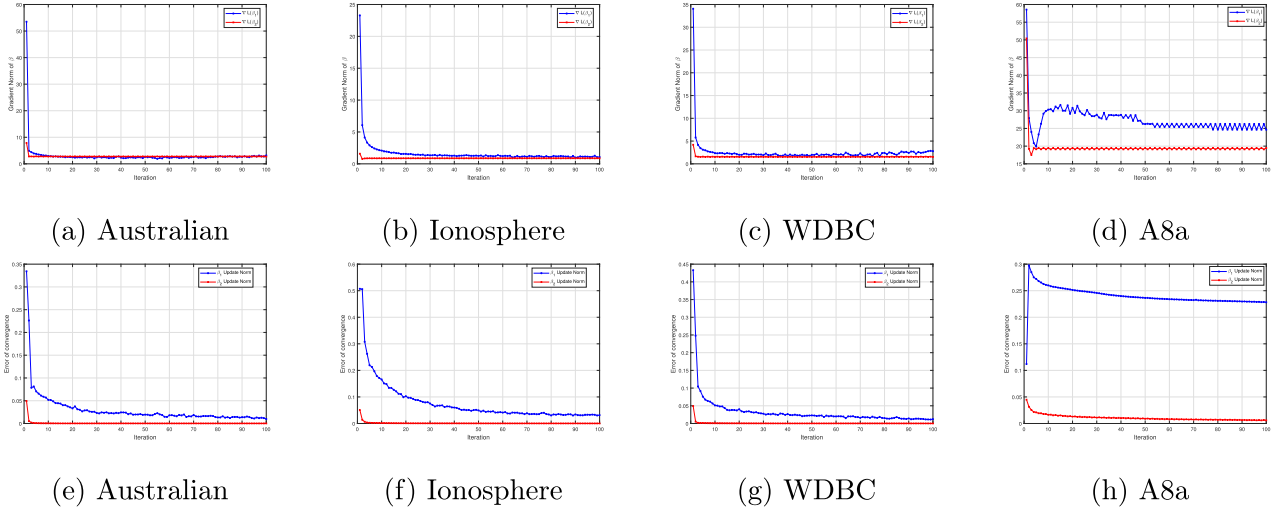
The results demonstrate that the algorithm exhibits significant sensitivity to the parameter  $p$ , with the classification ACC showing obvious fluctuation trends across different datasets as  $p$  varies. Take the Australian dataset as an example, where the variation range of its ACC metric is particularly prominent. Notably, the impacts of parameters  $a$ ,  $b$ , and  $C_i$  on the algorithm performance are relatively weak and limited. Parameter  $a$  only exerts an optimizing effect on ACC within specific value ranges of partial datasets; the influence of parameter  $b$  mainly concentrates in the interval from  $10^{-1}$  to  $10^2$ ; while the combined effect of parameter  $C_i$  only emerges in specific dataset scenarios. This highly concentrated characteristic of parameter sensitivity endows the PBLTELM algorithm with remarkable stability.

Overall, the PBLTELM algorithm demonstrates remarkable advantages. In practical application scenarios, users only need to focus on

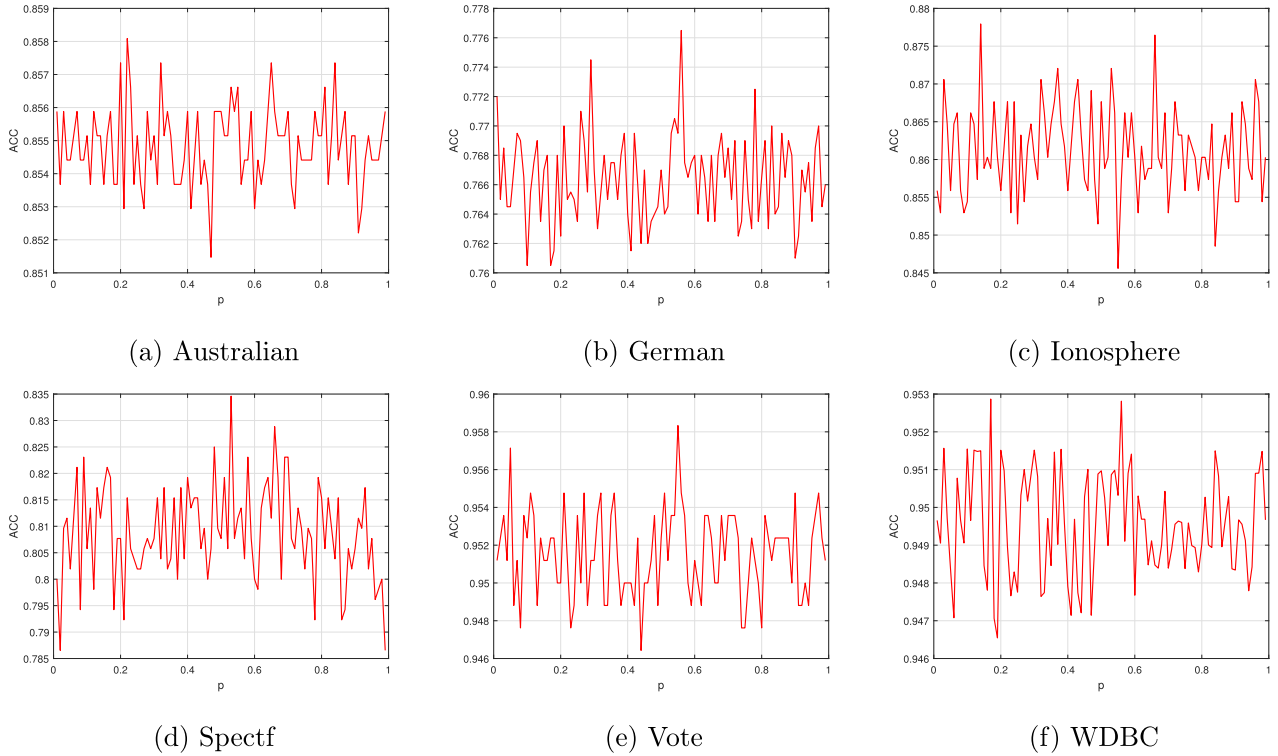
debugging the parameter  $p$  to achieve stable and efficient classification effects on different types of datasets. The low sensitivity of the remaining parameters effectively avoids performance fluctuations caused by slight parameter changes, greatly reduces the complexity and cost of parameter tuning, significantly enhances the engineering application value and generalization ability of the algorithm, and fully demonstrates the robust performance of the PBLTELM algorithm in complex data environments.

#### 4.6. Friedman test

Seven algorithms were evaluated using real datasets and image datasets. The results showed that the PBLTELM algorithm exhibited relatively high accuracy. To comprehensively analyze the differences in classification performance among the seven algorithms across different datasets, statistical analysis was conducted using the Friedman test in this study. As a non-parametric test method, the Friedman test is widely



**Fig. 16.** Convergence properties of the inner and outer layers of Algorithm 1: first Row for Inner Layer, second row for outer layer.



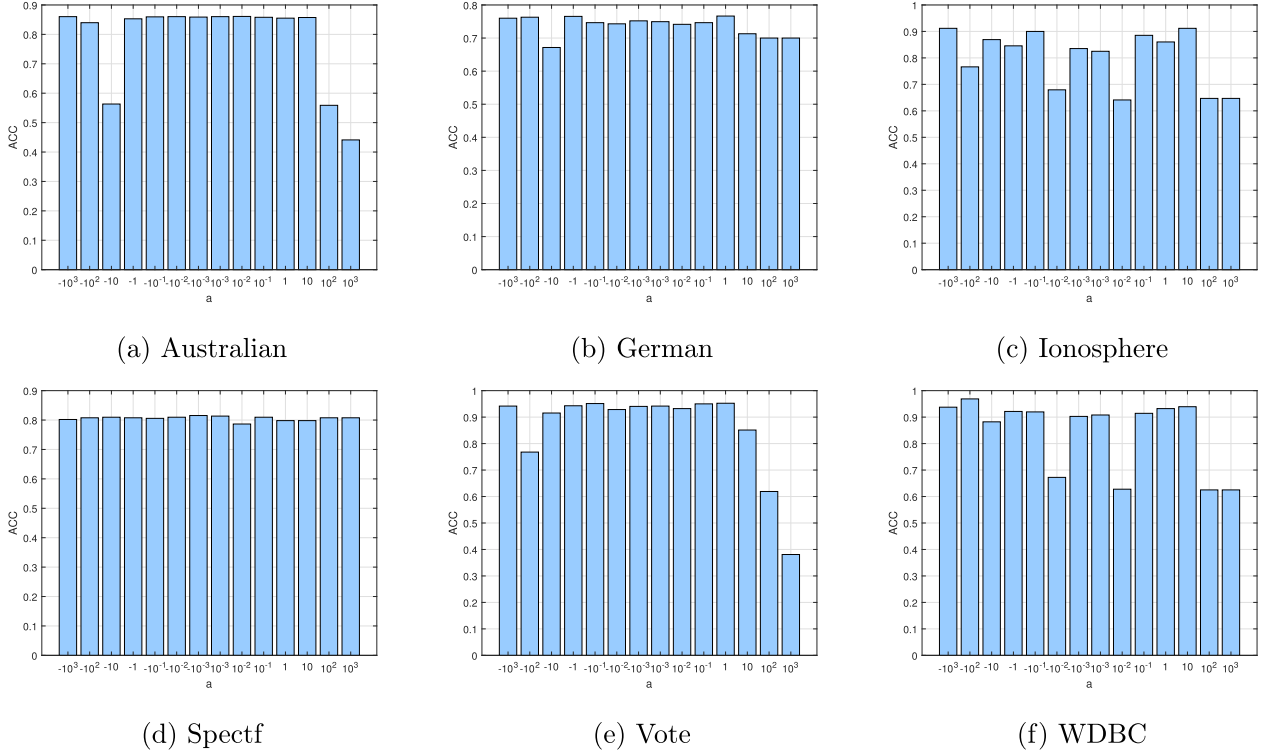
**Fig. 17.** Classification performance of PBLTELM with different Parameter  $p(C_i = 1(i = 1, 2, 3, 4), a = 1, b = 1)$ .

used in the field of machine learning performance evaluation due to its simplicity in operation and strong robustness. Its core principle involves assessing differences in algorithm performance by analyzing the ranking data of algorithms across different datasets.

In this study, the null hypothesis  $H_0$  of the Friedman test was set as follows: no significant differences exist in the classification performance among the seven algorithms. The alternative hypothesis  $H_1$ , in contrast, states that significant differences are present in the classification performance of the seven algorithms. The specific operational procedure is as follows: first, the algorithms were ranked based on their ACC across different datasets; subsequently, the Friedman statistic  $F_F$  was calculated a statistic that comprehensively captures the ranking differences between algorithms. The computed  $F_F$  value was then compared against the crit-

ical value associated with the corresponding degrees of freedom. If the  $F_F$  value exceeds the critical value, the null hypothesis is rejected, indicating that significant differences exist in the classification performance of the seven algorithms.

Although the PBLTELM algorithm exhibited overall favorable performance in the preliminary numerical experiments, its performance was not found to be optimal across all datasets. Therefore, as a rigorous statistical analysis method, the Friedman test is of crucial importance in verifying the significance of the performance advantages of the PBLTELM algorithm. Through this test, it can be scientifically determined whether there exist statistically significant differences in classification performance between the PBLTELM algorithm and the other six algorithms, thereby providing



**Fig. 18.** Classification performance of PBLTELM with different Parameter  $a(C_i = 1(i = 1, 2, 3, 4), b = 1, p = .1)$ .

a solid theoretical foundation for the effectiveness of the PBTELM algorithm.

The Friedman statistic is calculated as follows:

$$\chi^2 = \frac{12N}{p(p+1)} \left[ \sum_j R_j^2 - \frac{p(p+1)^2}{4} \right],$$

where  $N$  is taken to represent the number of datasets used in the experiments,  $p$  is defined as the number of algorithms employed, and  $R_j$  is used to denote the average ranking of the  $j$ -th algorithm across all datasets. The key term involved in the computation is  $\sum_j R_j^2 - \frac{p(p+1)^2}{4}$ , and this term comprises two components: first,  $\sum_j R_j^2$ , which is the sum of the squared average rankings of all algorithms and is used to reflect the degree of variability in algorithm rankings; second,  $\frac{p(p+1)^2}{4}$ , which represents the theoretical expectation under the null hypothesis. The  $F$ -distribution with degrees of freedom  $p - 1$  and  $(p - 1)(N - 1)$  is employed to compute the following content:

$$F_F = \frac{(N - 1)\chi^2}{N(p - 1) - \chi^2}.$$

The degrees of freedom of the numerator,  $(p - 1)$ , are used to reflect the dimensions of differences between the algorithms. The degrees of freedom of the denominator,  $(p - 1)(N - 1)$ , are obtained by combining the number of algorithms and datasets, and they serve to represent the error component of the overall data variation after accounting for the effects of algorithms. This enables the measurement of the impact of random factors on the results. After  $F_F$  is calculated, it is compared with the critical value corresponding to the relevant degrees of freedom retrieved from the  $F$ -distribution table. If  $F_F$  exceeds the critical value, the null hypothesis is rejected, which indicates that statistically significant differences in classification performance exist among the algorithms across the datasets.

In the experiments, the significant differences between the PBTELM algorithm and the other six algorithms were illustrated separately using two sources: the UCI dataset and the libsvm dataset. In the UCI dataset, where  $N = 12$  and  $p = 7$ , the obtained  $\chi^2$  values were 31.6, 38.8, and 30

respectively, and the  $F_F$  values were 8.61, 12.87, and 7.85 respectively when the noise levels were 0, 0.1, and 0.3. In the libsvm dataset, where  $N = 7$  and  $p = 7$ , the obtained  $\chi^2$  value and  $F_F$  value were 23.6 and 7.69 respectively when the noise level was 0. In exploratory studies, a relatively high value of  $\alpha$  (e.g.,  $\alpha = 0.05$ ) can enhance the test power, making it easier to detect potential differences between algorithms. At the significance level of  $\alpha = 0.05$ , the critical value of  $F(6, 66)$  is  $F_\alpha = 2.23$ , and the critical value of  $F(6, 36)$  is  $F_\alpha = 2.35$ . Apparently,

$$F_F = 8.61 > 2.23 = F_\alpha,$$

$$F_F = 12.87 > 2.23 = F_\alpha,$$

$$F_F = 7.85 > 2.23 = F_\alpha,$$

$$F_F = 7.69 > 2.35 = F_\alpha.$$

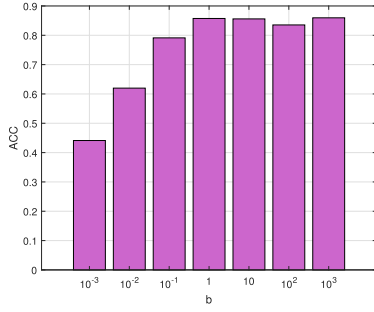
Therefore, the null hypothesis  $H_0$  is rejected, and the alternative hypothesis  $H_1$  is accepted. This means that significant differences exist among the six algorithms under comparison.

In order to further compare the performance of the seven algorithms, the Bonferroni-Dunn post hoc test was used to assess the significant differences among the algorithms. Based on the Studentized range statistic divided by  $\sqrt{2}$ ,  $q_{\alpha=0.05}(p)$  was set to 2.690, and the critical difference statistic (CD) was derived respectively for the UCI dataset and the libsvm dataset as follows:

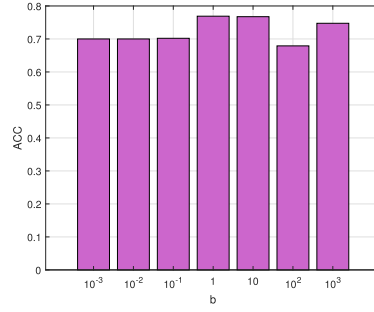
$$CD = q_{\alpha=0.05}(p) \times \sqrt{\frac{p(p+1)}{6N}} = 2.690 \times \sqrt{\frac{7 \times 8}{6 \times 12}} = 2.37,$$

$$CD = q_{\alpha=0.05}(p) \times \sqrt{\frac{p(p+1)}{6N}} = 2.690 \times \sqrt{\frac{7 \times 8}{6 \times 7}} = 3.1.$$

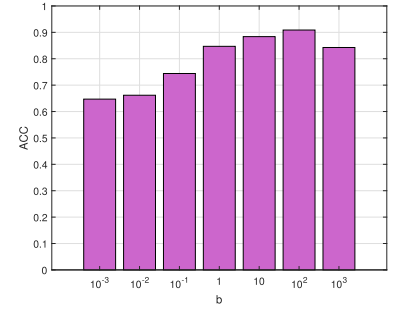
It indicates that in the UCI datasets, if the difference in average rankings between two algorithms is at least 2.37, the accuracy of one algorithm is significantly superior to that of the other. Similarly, in the libsvm dataset, if the difference in average rankings between two algorithms is at least 3.1, the accuracy of one algorithm is significantly superior to that of the other. As shown in Table 11, the PBTELM algorithm proposed in this study exhibits significant advantages over the other six algorithms. In the UCI dataset, the PBTELM algorithm is superior to the



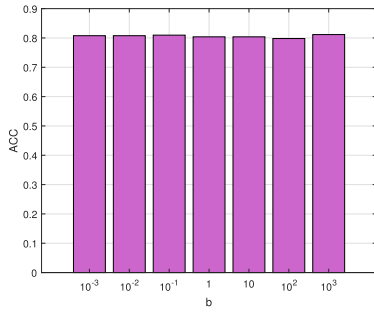
(a) Australian



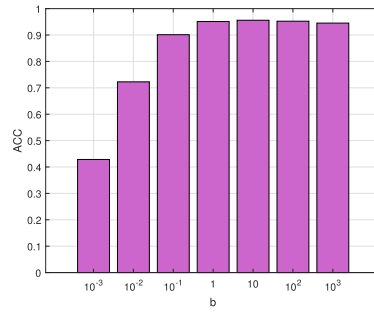
(b) German



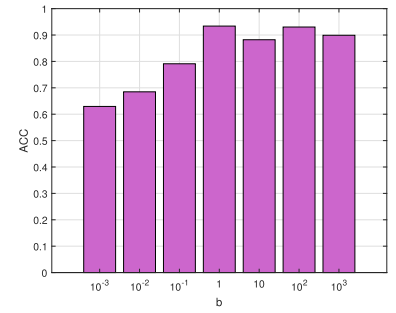
(c) Ionosphere



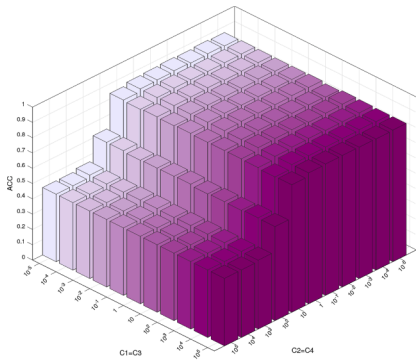
(d) Spectf



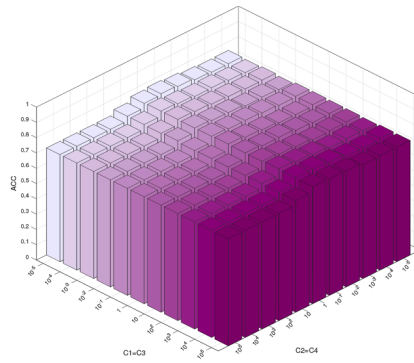
(e) Vote



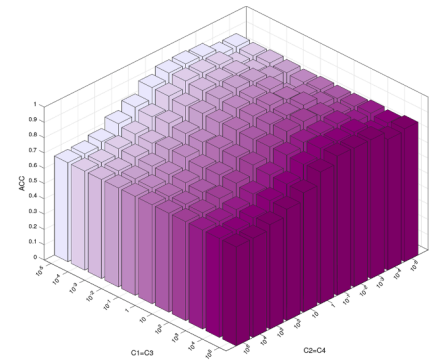
(f) WDBC

**Fig. 19.** Classification performance of PBLTELM with different Parameter  $b(C_i = 1(i = 1, 2, 3, 4), a = 1, p = .1)$ .

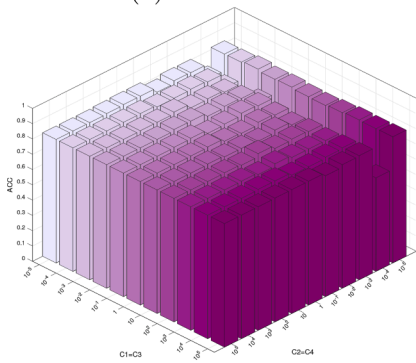
(a) Australian



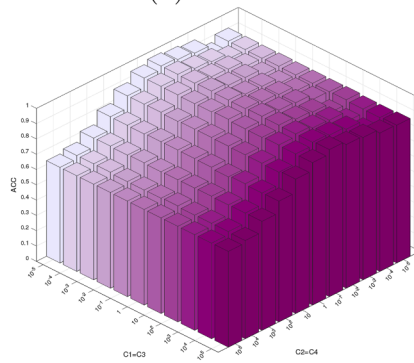
(b) German



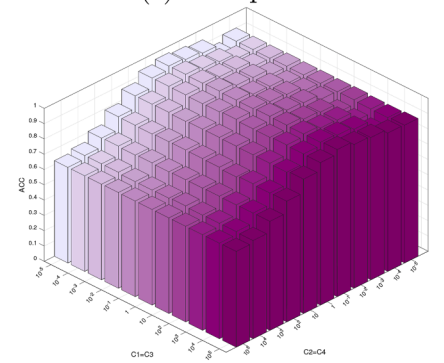
(c) Ionosphere



(d) Spectf

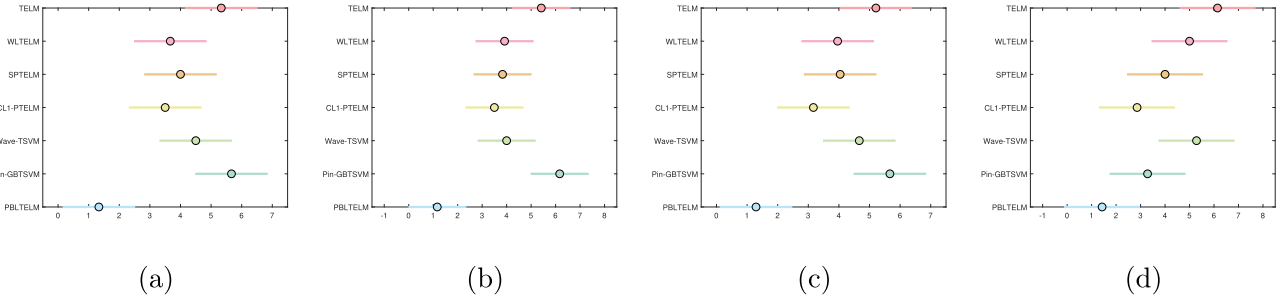


(e) Vote



(f) WDBC

**Fig. 20.** Classification performance of PBLTELM with different Parameter  $C_i(i = 1, 2, 3, 4)(a = 1, b = 1, p = .1)$ .



**Fig. 21.** Visualization of Post Hoc Tests for Real-World Datasets. ((a) to (d) respectively correspond to the cases of UCI datasets under no noise, 10% noise, 30% noise, and libsvm datasets under no noise).

**Table 11**  
Difference of Average Ranks on UCI Datasets and Libsvm Datasets.

| Model comparison                | UCI Datasets<br>Gaussian noise |      |      | Libsvm Datasets |
|---------------------------------|--------------------------------|------|------|-----------------|
|                                 | 0%                             | 10%  | 30%  | 0%              |
| d(PBTELM-TELM)                  | 4                              | 4.25 | 3.92 | 4.71            |
| d(PBTELM-WLTEL)                 | 2.33                           | 2.75 | 2.67 | 3.57            |
| d(PBTELM-SPTEL)                 | 2.67                           | 2.67 | 2.75 | 2.57            |
| d(PBTELM-CL <sub>1</sub> -PTEL) | 2.17                           | 2.33 | 1.92 | 1.43            |
| d(PBTELM-Wave-TSVM)             | 3.17                           | 2.83 | 3.42 | 3.86            |
| d(PBTELM-Pin-GBTSVM)            | 4.33                           | 5    | 4.42 | 1.86            |

other algorithms, while in the libsvm dataset, the PBTELM algorithm is superior to the TELM, WLTEL, and Wave-TSVM algorithms.

Based on the data in Table 11 and Fig. 21, for UCI datasets, the average rank of PBTELM is found to be relatively high in the noise-free scenario; after 10% noise is introduced, its rank interval remains in the low-rank region without significant fluctuations; even in the scenario with 30% high noise, the rank of PBTELM is slightly shifted backward, yet it still stays within a relatively high-rank interval. Meanwhile, regardless of changes in noise levels, a relatively obvious separation is consistently observed between the rank interval of PBTELM and those of algorithms such as TELM and Pin-GBTSVM, while a certain overlap is maintained between its rank interval and that of some algorithms like CL<sub>1</sub>-PTEL and Wave-TSVM. For the noise-free libsvm dataset scenario corresponding to Fig. 21(d), the rank of PBTELM is also skewed toward the low-rank region, which indicates that in these real-world dataset tasks, the classification performance of PBTELM is relatively better, and no significant difference is observed between its performance and that of some comparison algorithms.

## 5. Conclusion

In this paper, a novel TELM learning framework named  $L_p$  norm sparse Blinex Twin Extreme Learning Machine (PBTELM) was proposed. Compared with several classical advanced classification algorithms, the proposed PBTELM not only has anti-noise characteristics and sparsity, but also shows significant advantages in classification speed. The effectiveness of PBTELM is verified by numerical experiments on two-dimensional artificial dataset, CMU datasets, UCI benchmark datasets and libsvm relatively large-scale datasets. Experimental results show that the proposed model is superior to other comparison algorithms in classification accuracy and speed while maintaining sparsity and insensitivity to outliers.

Although significant advantages in sparsity, noise resistance, and computational efficiency have been demonstrated by the PBTELM model in relatively large-scale data classification tasks, several limitations that are worthy of in-depth exploration remain. One of the core theoretical foundations of the current model, Theorem 2, relies for its proof on the assumption that at least one of matrices  $U$  or  $V$  is of full

column rank; however, in high-dimensional and large-scale data scenarios, data matrices often struggle to meet the strict full-rank condition. In such cases, the effectiveness of Theorem 2 has not yet been fully verified, and with the further expansion of data dimension and scale, the associated computational cost may increase significantly, affecting the practical deployment efficiency of the model. Meanwhile, although the combination of  $L_p$ -norm sparsity and the Adam optimization method has endowed the model with excellent performance, the computational trade-off between the two in ultra-large-scale data processing still requires further exploration. For instance, the balance between sparse constraints and the iterative efficiency of optimization algorithms, as well as the complexity of parameter tuning when data volume surges, may all become factors limiting the scalability of the model.

Based on these limitations, future research can be deepened in multiple directions. Firstly, targeted research on extending the applicability of Theorem 2 needs to be conducted for scenarios involving non-strict full-rank data matrices commonly encountered in high-dimensional data. Through supplementary theoretical derivations and extensive empirical analyses, its effectiveness under complex data distributions should be verified, and targeted computational optimization schemes should be designed to reduce the computational overhead in high-dimensional scenarios. Secondly, focusing on the core mechanism of the combination of  $L_p$ -norm sparsity and Adam optimization, more efficient iterative strategies and parameter update rules should be explored to further optimize the computational trade-off, enhance the scalability of the model on ultra-large-scale datasets, while maintaining the storage and speed advantages brought by sparsity. Furthermore, the Bayesian search strategy should be introduced into the hyperparameter optimization process of the model, and intelligent search algorithms should be used to quickly find the optimal parameter combination suitable for large-scale data, thereby improving the training efficiency and generalization performance of the model in massive data scenarios. In addition, the optimized PBTELM framework can be further extended to fields such as multi-view, multi-instance, and multi-label learning, as well as weakly supervised and semi-supervised learning. By leveraging its sparse and noise-resistant characteristics, it can be adapted to more complex practical application scenarios such as medical big data, smart city sensing data, and massive financial risk control data. Meanwhile, the potential of the model in cross-domain transfer learning should be explored to further broaden its application boundaries and enhance the theoretical depth and practical application value of the research outcomes.

## Author Contributions

**TianTian Jiang, Guolin Yu & Jun Ma :** The responsibilities included algorithm development, software creation, numerical example preparation, original draft writing, and review and editing of the manuscript. All authors contributed equally to this work and have read and approved the final version of the manuscript for publication.

## Use of AI Tools Declaration

The authors declare they have not used Artificial Intelligence (AI) tools in the creation of this article.

## Conflict of Interest

The authors declare that they have no conflict of interest and no relevant financial or non-financial interests to disclose.

## CRediT authorship contribution statement

**Tiantian Jiang:** Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Data curation, Conceptualization; **Guolin Yu:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization; **Jun Ma:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization.

## Declaration of competing interest

We declare that we have no financial and personal relationships with other people or organizations that can inappropriately influence our work, there is no professional or other personal interest of any nature or kind in any product, service and/or company that could be construed as influencing the position presented in, or the review of, the manuscript entitled “Fast Sparse Supervised Learning Framework with BLInex Loss Function”.

## Acknowledgments (All Sources of Funding of The Study Must Be Disclosed)

This research was supported in part by the [National Natural Science Foundation of China](#) (No.62366001, No.12361062), in part by the [Natural Science Foundation of Ningxia Province](#) (No.2024AAC05055, No.2023AAC02053), in part by the Fundamental Research Funds for the Central Universities of North Minzu University (No.2023ZRLG01) and in part by the Postgraduate Innovation Project of North Minzu University (YCX24261).

## References

- Ahmed, M. A., Deyu, Q., & Alshemmary, E. N. (2020a). Electroencephalogram signal eye blink rejection improvement based on the hybrid stone blind origin separation and particle swarm optimization technique. *IEEE Access*, 8, 105671–105680.
- Ahmed, M. A., Qi, D., & Alshemmary, E. N. (2020b). Effective hybrid method for the detection and rejection of electrooculogram (eog) and power line noise artefacts from electroencephalogram (eeg) mixtures. *IEEE Access*, 8, 202919–202932.
- Akhtar, M., Tanveer, M., & Arshad, M. (2024). Advancing supervised learning with the wave loss function: A robust and smooth approach. *Pattern Recognition*, 155, 110637. Alzheimer's Disease Neuroimaging Initiative.
- Alshemmary, E. N., Hilal, B. K., Abdulbaqi, A. S., Ahmed, M. A., & Lu, Z. (2024). Eeg eye blink artifacts removal with wavelet denoising and bandpass filtering. *Baghdad Science Journal*, 21(11), 8.
- Anand, P., Bharti, A., & Rastogi, R. (2023). Time efficient variants of twin extreme learning machine. *Intelligent Systems with Applications* 17, 200169.
- Bartlett, P. L., Jordan, M. I., & McAuliffe, J. D. (2006). Convexity, classification, and risk bounds. *Journal of the American Statistical Association*, 101(473), 138–156.
- Ding, S., Yu, J., Qi, B., & Huang, H. (2014). An overview on twin support vector machines. *Artificial Intelligence Review*, 42, 245–252.
- Ding, S., Zhang, N., Zhang, X., & Wu, F. (2017). Twin support vector machine: Theory, algorithm and applications. *Neural Computing and Applications*, 28, 3119–3130.
- Dufrenois, F., & Noyer, J. C. (2015). Generalized eigenvalue proximal support vector machines for outlier description. In *International joint conference on neural networks* (pp. 1–9). IEEE.
- Fung, G. M., & Mangasarian, O. L. (2011). Equivalence of minimal  $\ell_0$ - and  $\ell_p$ -norm solutions of linear equalities, inequalities and linear programs for sufficiently small  $p$ . *Journal of Optimization Theory and Applications*, 151(1), 1–10.
- Huang, G. B., Zhu, Q. Y., & Siew, C. K. (2004). Extreme learning machine: A new learning scheme of feedforward neural networks. In *International joint conference on neural networks* (pp. 985–990). IEEE.
- Huang, X., Shi, L., & Suykens, J. A. (2013). Support vector machine classifier with pinball loss. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 36(5), 984–997.
- Moosaei, H., & Hladik, M. (2023). Sparse solution of least-squares twin multi-class support vector machine using  $l_0$  and  $l_p$ -norm for classification and feature selection. *Neural Networks*, 166, 471–486.
- Olewi, Z. C., Alshemmary, E. N., & Al-Augby, S. (2024). Developing hybrid cnn-gru arrhythmia prediction models using fast fourier transform on imbalanced. *ECG Datasets. Mathematical Modelling of Engineering Problems*, 11(2), 413–429.
- Quadir, A., & Tanveer, M. (2024). Granular ball twin support vector machine with pinball loss function. *IEEE Transactions on Computational Social Systems*, 12(5), 3891–3900.
- Ren, Z., & Yang, L. (2018). Correntropy-based robust extreme learning machine for classification. *Neurocomputing*, 313, 74–84.
- Sharma, S., Rastogi, R., & Chandra, S. (2021). Large-scale twin parametric support vector machine using pinball loss function. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 51(2), 987–1003.
- Shen, J., & Ma, J. (2019). Sparse twin extreme learning machine with  $\epsilon$ -insensitive zone pinball loss. *IEEE Access*, 7, 112067–112078.
- Si, Q., Yang, Z., & Ye, J. (2023). Symmetric linex loss twin support vector machine for robust classification and its fast iterative algorithm. *Neural Networks*, 168, 143–160.
- Tang, J., Li, J., Xu, W., Tian, Y., Ju, X., & Zhang, J. (2021). Robust cost-sensitive kernel method with blinex loss and its applications in credit risk evaluation. *Neural Networks*, 143, 327–344.
- Tanveer, M., Gupta, T., & Shah, M. (2021). Pinball loss twin support vector clustering. *ACM Transactions on Multimedia Computing, Communications, and Applications*, 17(2s), 1–23. Alzheimer's disease neuroimaging initiative.
- Tanveer, M., Rajani, T., & Ganaie, M. A. (2019a). Improved sparse pinball twin SVM. *International conference on systems, man and cybernetics*. IEEE, 3287–3291.
- Tanveer, M., Tiwari, A., Choudhary, R., & Ganaie, M. A. (2022). Large-scale pinball twin support vector machines. *Machine Learning*, 111(10), 3525–3548.
- Tanveer, M., Tiwari, A., Choudhary, R., & Jalan, S. (2019b). Sparse pinball twin support vector machines. *Applied Soft Computing*, 78, 164–175.
- Tian, Y., Ju, X., & Qi, Z. (2014). Efficient sparse nonparallel support vector machines for classification. *Neural Computing and Applications*, 24(5), 1089–1099.
- Vapnik, V. N. (1999). An overview of statistical learning theory. *IEEE Transactions on Neural Networks*, 10(5), 988–999.
- Wan, Y., Song, S., Huang, G., & Li, S. (2017). Twin extreme learning machines for pattern classification. *Neurocomputing*, 260, 235–244.
- Wang, H., Hong, X., & Zhang, S. (2024a). Twin bounded support vector machine with capped pinball loss. *Cognitive Computation*, 16(5), 2185–2205.
- Wang, H., & Shao, Y. (2023a). Fast truncated huber loss svm for large scale classification. *Knowledge-Based Systems*, 260, 110074.
- Wang, H., & Shao, Y. (2023b). Sparse and robust svm classifier for large scale classification. *Applied Intelligence*, 53(16), 19647–19671.
- Wang, H., & Shao, Y. (2024). Fast generalized ramp loss support vector machine for pattern classification. *Pattern Recognition*, 146, 109987.
- Wang, H., Yu, G., & Ma, J. (2024b). Fast sparse twin learning framework for large-scale pattern classification. *Engineering Applications of Artificial Intelligence*, 130, 107730.
- Xing, H. J., Wang, S. W., He, Z. C., & Wang, Y. D. (2026). Robust least squares twin support vector machine with adaptive pinball loss. *IEEE Transactions on Neural Networks and Learning Systems*.
- Yan, H., Ye, Q. L., & Yu, D. J. (2019). Efficient and robust twsvm classification via a minimum l1-norm distance metric criterion. *Machine Learning*, 108, 993–1018.
- Yang, Y., Xue, Z., Ma, J., & Chang, X. (2023). Robust projection twin extreme learning machines with capped l1-norm distance metric. *Neurocomputing*, 517, 229–242.
- Yu, G., Ma, J., & Xie, C. (2023). Hessian scatter regularized twin support vector machine for semi-supervised classification. *Engineering Applications of Artificial Intelligence*, 119, 105751.
- Yuan, C., & Yang, L. (2021). Robust twin extreme learning machines with correntropy-based metric. *Knowledge-Based Systems* 214, 106707.
- Zhang, Y., & Ye, W. (2017). Sparse recovery by the iteratively reweighted algorithm for elastic minimization. *Optimization*, 66(10), 1677–1687.
- Zhang, Z., Zhen, L., Deng, N., & Tan, J. (2014). Sparse least square twin support vector machine with adaptive norm. *Applied Intelligence*, 41(4), 1097–1107.
- Zheng, X., Zhang, L., & Yan, L. (2022). Sparse discriminant twin support vector machine for binary classification. *Neural Computing and Applications*, 34(19), 16173–16198.